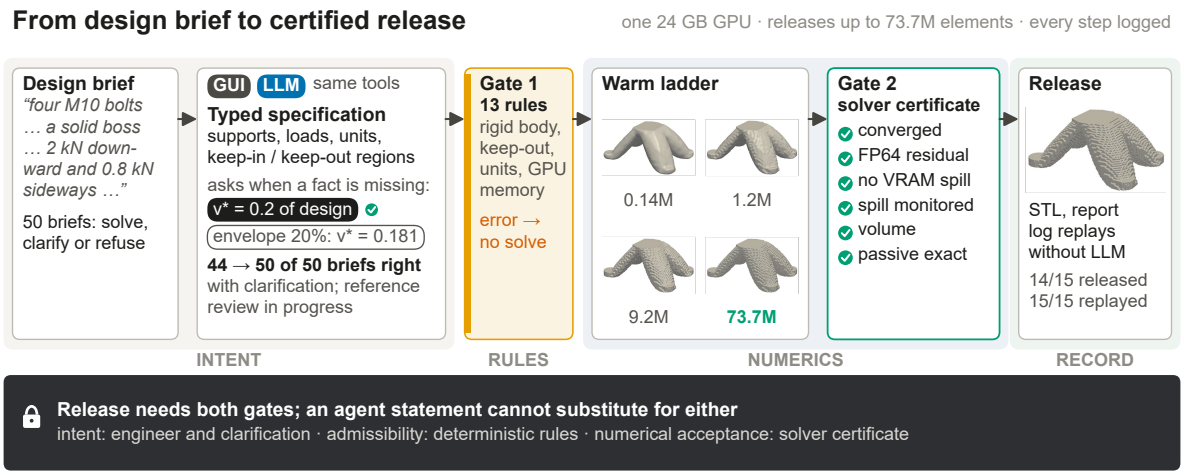
GUIDEBOOK · Version 0.0.1

AgenticTO

Agent-assisted, certified 3D topology optimization on one GPU: the workspace, the tool contract and the release record



Shaoliang Yang **Jun Wang**

Department of Mechanical Engineering, Santa Clara University

Companion to the paper

From topology-optimization brief to solver-checked release: separating intent, admissibility and acceptance knowledge for language agents

October 2026

Software version 0.0.1, spec schema `agentto.spec/v0`

AgenticTO Guidebook
Version 0.0.1, October 2026. Describes AgenticTO 0.0.1 (Python package `agentto`) and the specification schema `agentto.spec/v0`.

Authors. Shaoliang Yang and Jun Wang, Department of Mechanical Engineering, Santa Clara University, 500 El Camino Real, Santa Clara, CA 95053, USA. Correspondence: Jun Wang, jwang22@scu.edu.

Software. The application code, the 50-brief task suite with its adjudicated reference readings, the saved agent outputs and tool logs, the numerical result records, the generators of the tables and data figures, and the `voxfem` solver that AgenticTO runs on (BSD 3-Clause; Chapter 2) are available at `https://github.com/nbbllxx0/From-brief-design-to-verified-release`.

How this book is made. The reference tables for the specification schema, the rules, the tools, the certificate, the schedules, the command-line flags, the HTTP endpoints, the keyboard map and the script catalogue are generated from the source code by `docs/guidebook/gen_tables.py`, so they match the code they describe. Every screenshot comes from a real session recorded by `scripts/guide_shots.py`, which drives the workspace with real clicks and keystrokes. Numbers about performance and accuracy are taken from the paper or from the result files it cites, never measured again for this book; each such number is marked with its claim identifier from `results/claims_ledger.md`, for example [C3.1].

Scope. AgenticTO is research software. A certificate in this book means a set of numerical acceptance checks on a stated computation. It is not a structural safety assessment, and a released design needs further engineering analysis before anything is built from it.

Typography. Set in XCharter, Helvetica and Inconsolata. Colours follow the paper's figures: green for a passing check, vermillion for a failure, blue for the agent, amber for the rule gate, dark grey for the workspace.

Preface

A topology optimization can converge and still solve the wrong problem. A support may restrain a direction the engineer meant to leave free, a load may sit on the wrong face, a pad may be retained that nobody asked for. Neither the solver's residual nor a language agent's confident summary reveals such errors. AgenticTO is built around that observation. It keeps the design intent in one typed specification, checks it with deterministic rules before anything is solved, and lets only the solver issue the certificate that marks a result as released. A language agent can help at every step, but it works through the same logged tools as the engineer and cannot write a certificate.

This guidebook explains how to install the software, build and solve designs in its graphical workspace, work with the built-in assistant, connect other agents through the Model Context Protocol, read the provenance log, and reproduce the results of the companion paper.

Who this book is for. Two groups of readers. Journal reviewers who want to know what the software actually does, which check is responsible for which guarantee, and how each number in the paper can be traced to a file and a script: Chapters 1, 7, 11 and 12 are written for you. Engineers who download the public release and want to use it: start with Chapters 2 and 3, then read the chapters on the workspace, the specification and the rules.

How to read it. Chapter 1 gives the concepts in a few pages. Chapter 3 takes you from a fresh start to a certified, exported design in about ten minutes. The remaining chapters can be read in any order. The appendices are reference material: every field of the specification, every tool with its arguments, every certificate field, the keyboard map, every script, and a glossary. A one-page quick reference precedes the index.

Conventions.

- Interface labels are set in bold sans serif: **Optimize release**. Keys look like S or Ctrl+K.
- File names, commands, tool names and JSON fields are set in monospace: `scripts/serve.py`, `validate_spec`, `volume_fraction`. Tool names link to their reference entry in Appendix B; rule codes such as **R003** link to Chapter 6.
- Commands are shown for a POSIX shell. On Windows, the same commands work in PowerShell except where a box says otherwise.
- A grey bracket such as [C2.3] names the claim in `results/claims_ledger.md` that backs a number.

> **Four kinds of boxes.** Green boxes like this one give practical tips. Blue boxes add background. Vermillion boxes warn about something that can cost you a result or mislead you. Amber boxes quote evidence from the paper, with its denominator.

Acknowledgement of tools. The software was developed with the help of OpenAI Codex and Claude-based coding tools, as stated in the paper's declaration. This guidebook was

drafted with AI assistance from the source code, the existing user guide and the paper; its reference tables are generated from the code.

Contents

Preface **iii**

1 What AgenticTO is **1**

1.1 Three checks, three kinds of knowledge 1

1.2 The five objects 2

1.3 Who may do what 3

1.4 How the software fits together 3

2 Installation and requirements **6**

2.1 Requirements 6

2.2 The Python environment 7

2.3 The web front end 7

2.4 Connecting the solver 7

2.5 Checking the installation 8

2.6 Optional: language agents on a subscription 8

3 Quick start: a certified release in ten minutes **9**

3.1 The brief 9

3.2 Start the workspace 9

3.3 Read the specification against the brief 10

3.4 Climb the ladder 10

3.5 Read the certificate 11

3.6 Export 12

3.7 On a smaller GPU 12

3.8 The same task with an agent 13

4 The workspace **14**

4.1 The areas 14

4.2 Status indicators 14

4.3 First launch 14

4.4 Moving around the 3D view 15

4.5 The timeline 15
4.6 Themes, keyboard and palette 15

5 Building a specification 18
5.1 Tutorial: a cantilever bracket 18
5.2 The document 20
5.3 Editing through the tools 24

6 Rules 26
6.1 When the rules run 26
6.2 Reading a finding 26
6.3 Summary 27
6.4 Rule by rule 27
6.5 What the rules cannot see 29

7 The resolution ladder and the release certificate 31
7.1 The rungs 31
7.2 Warm starts 32
7.3 Release policies 32
7.4 The release certificate 33
7.5 Surface, voxels and stale results 35
7.6 Export 35

8 Working with the assistant 37
8.1 What the assistant can do 37
8.2 Propose and Act 37
8.3 Clarification questions with consequence previews 38
8.4 Cards and the ghost overlay 39
8.5 An Act run 39
8.6 When the assistant helps, and what it costs 40

9 Connecting external agents through MCP 43
9.1 The server 43
9.2 Profiles 44
9.3 Attach Claude Code to the workspace 44
9.4 Attach the Codex CLI 45
9.5 Stand-alone sessions 46

9.6 Conventions an agent needs 46
9.7 The HTTP interface 47

10 Logs, replay and provenance 49
10.1 The session folder 49
10.2 Records 49
10.3 The specification hash 50
10.4 Replay without a language model 51
10.5 The tool contract from Python 51

11 Reproducing the paper 53
11.1 Three levels 53
11.2 Rebuilding the tables, figures and ledger 53
11.3 Main paper 54
11.4 Supplementary material 55
11.5 The claims ledger 55
11.6 Re-running the studies 56

12 Limits and troubleshooting 58
12.1 Scope of the software 58
12.2 What the evidence does not establish 58
12.3 Troubleshooting 59
12.4 Reporting a problem 60

A Specification reference 61

B Tool reference 66

C Certificate, report and log fields 73

D Keyboard, mouse and command palette 76

E Script catalogue 78

F Glossary 83

G Quick reference 86

Index 87

1

What AgenticTO is

AgenticTO turns a written design brief into a certified 3D topology-optimized part. This chapter explains the three independent checks the system is built around, the five objects you will meet everywhere, who may do what, and how the parts of the software fit together.

AgenticTO minimizes the compliance of a structure (maximizes its stiffness) under a material budget, on a regular voxel grid, with the density-based SIMP method and a GPU multigrid solver. You describe the problem: a box of space, its supports and loads, regions that must stay solid or empty, a material and a volume fraction. AgenticTO solves it on a ladder of ever finer grids, up to several times 10^7 elements on one 24 GB graphics card, and exports the result as a closed STL surface with a report.

What sets the system apart is not the solver, which comes from companion work, but the way it separates the questions that have to be answered before a result can be trusted. A design can fail in three different ways, and each needs its own check.

1.1 Three checks, three kinds of knowledge

Figure 1.1 shows the path from a brief to a released design.

Intent. Does the specification say what the engineer meant? Only the engineer can answer this, so AgenticTO makes the specification readable and editable, shows every region in the viewport, and lets the assistant ask a question when a brief leaves a design-changing fact open (Chapter 8).

Formulation. Is the stated problem admissible at all? Thirteen deterministic rules check it within milliseconds of every edit: free rigid-body motion, regions that select nothing, a load inside a keep-out region, overlapping keep-in and keep-out regions, a grid too large for the GPU, and more (Chapter 6). Any error blocks solving.

Numerics. Did the computation meet its own requirements? The solver issues a certificate with six predicates: convergence, an accepted FP64 state solve, no GPU memory spill, active spill monitoring, the volume within 10^{-3}, and exact keep-in and keep-out regions (Chapter 7). Only a certified result on the release rung is a released design.

The three checks are not redundant. Each catches failures the others pass, and the paper shows this with experiments.

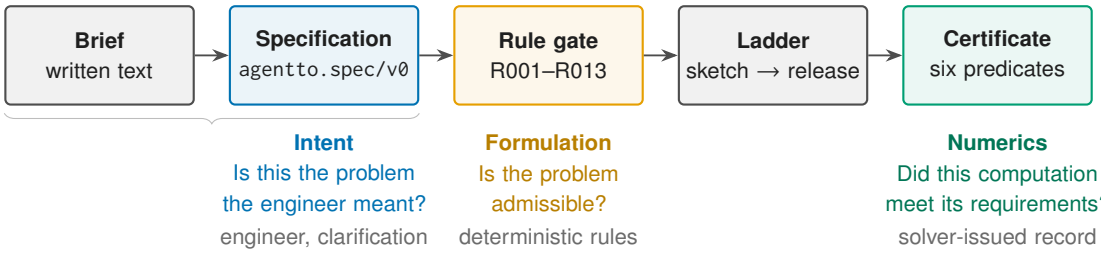


Figure 1.1. The path from a brief to a released design, and the question each check answers. A design is released only when all three are settled: the engineer (helped by clarification) settles intent, the rules settle admissibility, and the solver's certificate settles numerical acceptance.

Each check catches what the others miss.

- **Intent errors pass the rules.** With grounding tools but without questions, the primary agent returned 6 wrong specifications on 50 briefs, and all 6 passed every rule. With clarification it returned none [C2.3, C4.2].
- **A rule violation can pass the certificate.** With the rule gate switched off, a brief whose load sits inside its own keep-out region converged and received a passing numerical certificate, at about 3.8×10^8 times the compliance of the corrected problem. Rule **R008** stops it before any solve [C4.4].
- **The certificate holds back failed computations.** Of 54 recorded release attempts, 52 were certified; the census also contains 11 solves that ended without converging, which a system without the gate would have shown as designs [C4.3].

1.2 The five objects

Five objects run through the whole system. Every chapter of this book is about one or more of them.

The specification. One typed, versioned JSON document (`agentto.spec/v0`) holds the design intent: units, domain, material, volume fraction, supports, loads, keep-in and keep-out regions, base mesh and an optional minimum feature size. The workspace, the assistant, external agents and scripts all edit this same document through the same tools. It is geometric, not discrete: a face stays a face when the mesh changes (Chapter 5).

The rules. Thirteen checks with stable codes R001–R013 run on every change. Errors block solving; warnings and notes stay visible with a suggested fix (Chapter 6).

The ladder. Each problem is solved on up to four rungs (sketch, preview, refine, release). Each rung has twice as many elements along every edge as the one below, so eight times as many in total, and starts from the design of the rung below (Chapter 7).

The certificate. Every finished solve carries a certificate created by the solver. No tool can write one. Only a certified result on the release rung is a finished design.

The log. Every edit, agent step, answer and solve is written to `session.jsonl` with the actor who made it (user, agent, system or script) and the hash of the specification after it. A session can be replayed without any language model, and the solves reproduce bit for bit (Chapter 10).

Table 1.1. Who may do what. "Instruction" marks a limit stated in the assistant's instructions rather than enforced by the tool profile.

	Edit spec	Ask user	Solve sketch, preview	Solve refine, release	Write certificate	Notes
Engineer in the workspace	✓	–	✓	✓	–	starts every release in the GUI
Assistant, Propose	private copy	✓	–	–	–	changes come back as cards to accept
Assistant, Act	✓	✓	✓	instruction: no	–	a whole run can be undone in one step
External agent, `full` profile	✓	✓	✓	✓	–	every call tagged `agent` in the log
Solver	–	–	–	–	✓	from solver evidence only

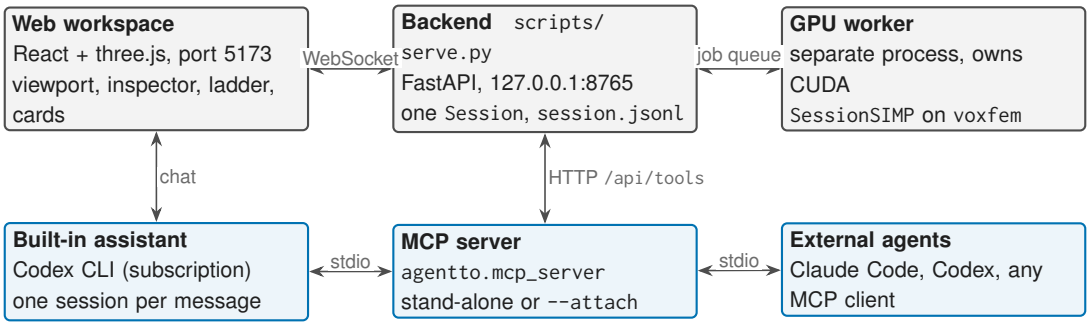


Figure 1.2. Processes and connections. All edits, whether from the workspace, the assistant or an external agent, become calls on the one `Session` in the backend and land in the same log. Solves run one at a time on the GPU worker.

1.3 Who may do what

AgenticTO makes authority explicit. Table 1.1 lists what each participant can do. The agent can propose, edit, ask and solve, but the certificate comes only from the solver, and the export of a released design requires a certified release-rung result.

> **A certificate is not a safety assessment.** The certificate states that a specific computation met specific numerical requirements on a stated grid. It does not prove a global optimum, and it says nothing about stress, buckling, fatigue, manufacturability or the behaviour of the exported surface as a physical part. Treat a released design as the input to further engineering analysis.

1.4 How the software fits together

Figure 1.2 shows the processes. The backend holds one `Session`, the object that owns the specification, its undo history, the solve jobs and the log. Every change, whatever its source, is a call of the form `Session.call(name, args, actor)`: the arguments are validated against the tool's signature, the tool runs, and one record is appended to `session.jsonl`.

The workspace sends its calls over a WebSocket, an attached agent over HTTP. The GPU worker is a separate process, so a solver failure or a memory spill cannot take the server down and the compiled kernels stay warm between jobs.

The tool contract has 27 tools (Appendix B). The same functions serve the workspace, the built-in assistant, external agents over MCP, and Python scripts.

Figure 1.3 follows one complete agent session through all of these pieces, from the brief to the exported release. Chapter 3 repeats the same task by hand in the workspace.

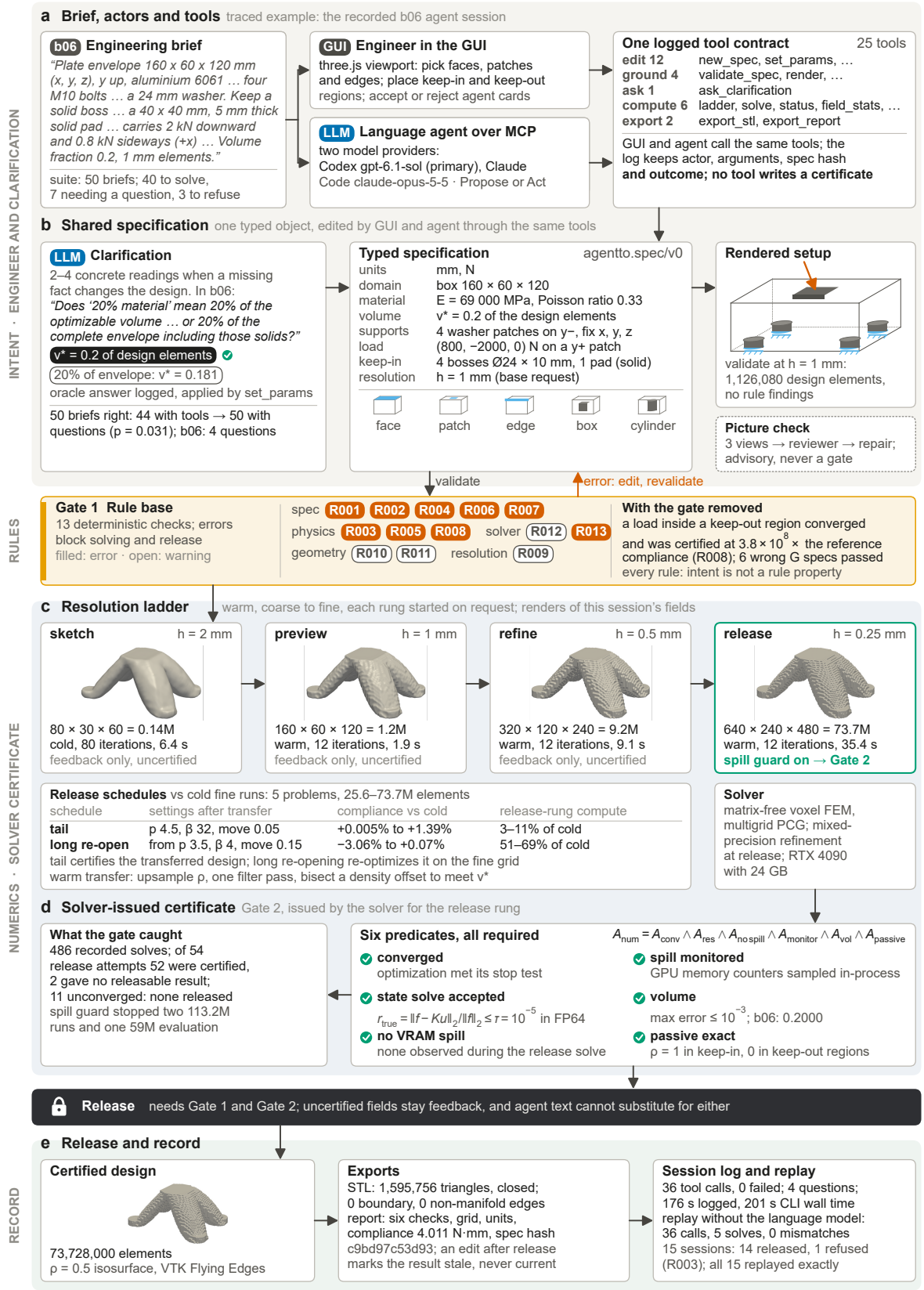


Figure 1.3. The AgenticTO workflow traced through one recorded agent session on the motor-mount brief b06, reproduced from Figure 1 of the paper. The engineer and clarification settle intent, the rule gate admissibility and the solver certificate numerical acceptance. Renders are $\rho = 0.5$ isosurfaces of the saved fields; under the tail schedule the fine rungs keep the transferred voxel steps.

2

Installation and requirements

AgenticTO is a Python package with a web front end and a separate GPU solver. This chapter lists what you need, installs the Python environment and the front end, connects the `voxfem` solver, and sets up the optional language agents, which run on personal subscriptions only.

2.1 Requirements

Table 2.1. Requirements. "Tested with" is the configuration used for every result in the paper.

Item	Tested with	Notes
GPU	NVIDIA GeForce RTX 4090, 24 GB	A CUDA 12 GPU. GPU memory bounds the finest grid you can release; rule **R013** estimates the memory before a solve starts.
Host	Intel Core i9-13900KF, 96 GB RAM	Large releases keep density fields in host memory as well.
Operating system	Windows 11	The spill guard reads the Windows GPU-memory counters. Other platforms have not been tested.
Python	3.10 or newer (a conda environment)	See Section 2.2 for packages.
Node.js	22	Only for the web front end.
Solver	`voxfem`	Included in `src/voxfem/` (BSD 3-Clause; Section 2.4).
Browser	Chrome	The workspace has been tested in Chrome with one user per session.
Codex CLI (optional)	0.165.0, ChatGPT login	Runs the built-in assistant.
Claude Code (optional)	2.1.312, claude.ai login	An external agent over MCP.

Shared desktop GPU. In the paper the GPU also drove the desktop, and other processes held about 2–5 GB of its memory. That is why the default ladder stops below 8×10^7 elements, although a 24 GB card could hold more on its own. A guarded attempt at 1.13×10^8 elements was stopped twice by the spill guard and is not part of any reported result [C3.2].

2.2 The Python environment

Create a conda environment with a CUDA 12 build of CuPy and the packages AgenticTO imports:

```
conda create -n agentto python=3.11
conda activate agentto
pip install numpy scipy matplotlib "cupy-cuda12x" "pydantic>=2.8,<3" \
    fastapi uvicorn mcp vtk pillow websockets pytest
```

What each package is for: `numpy`, `scipy` and `cupy` for the numerics and `field_stats`; `pydantic` for the specification schema and the tool arguments; `fastapi`, `uvicorn` and `websockets` for the backend; `mcp` for the agent interface; `matplotlib` for the setup pictures of the `render` tool; `vtk` for the marching-cubes surface and the STL export; `pillow` for thumbnails.

Optional extras:

- `pywin32` on Windows. The spill guard then reads the GPU memory counters in-process (about 0.1 ms per sample) instead of through a PowerShell subprocess (about 2.4 s per sample).
- `pyvista` for the 3D renders of the paper figures, `scikit-fem` for the independent cross-check of study X0, and `openpyxl` for the adjudication sheets (Chapter 11).

AgenticTO is not installed into the environment. The scripts put `src/` on the Python path themselves, and the MCP configurations in Chapter 9 set `PYTHONPATH`. If you prefer an installed package, `pip install -e .` in the repository root uses `pyproject.toml`.

2.3 The web front end

```
cd web
npm install
cd ..
```

This installs React, three.js and Vite at the versions pinned in `web/package.json`.

2.4 Connecting the solver

AgenticTO runs on `voxfem`, the GPU solver of the authors' companion work on fused matrix-free operators, Galerkin multigrid and guarded state solves. `voxfem` ships with the AgenticTO repository in `src/voxfem/`, unchanged and under its own BSD 3-Clause License, and is found automatically when `src/` is on the path. AgenticTO imports it read-only, never writes into it (no bytecode caches either), and extends it only by subclassing.

To use another copy of the solver, point AgenticTO at the directory that contains that `voxfem` package (the directory you would put on `PYTHONPATH` to make `import voxfem` work):

```
# POSIX shell
export VOXFEM_SRC=/path/to/other/src
# Windows PowerShell
$env:VOXFEM_SRC = "D:/other/src"
```

Set the variable in the shell that starts the backend, the MCP server or a script; the GPU worker inherits it. The older name `AGENTTO_VOXFEM_SRC` still works.

Without a GPU. Without a CUDA GPU you can still read the code, the briefs, the logs and every result file, and regenerate the paper's tables and data figures from those files (Chapter 11). Every solve needs a CUDA GPU.

2.5 Checking the installation

```
python -B -m pytest -q
```

The tests that solve on the GPU need a free GPU. Run one GPU job at a time: another CUDA program on the same card can push a solve into shared memory, and the spill guard will then stop it.

2.6 Optional: language agents on a subscription

AgenticTO calls language models only through command-line tools that are logged in with a personal subscription: the Codex CLI with a ChatGPT login for the built-in assistant, and optionally Claude Code with a claude.ai login.

```
codex login        # sign in with your ChatGPT account
claude             # the first run asks you to sign in with your claude.ai account
```

Before every call AgenticTO checks two things, and refuses to start the agent if either fails:

1. No API-key variable is set: `OPENAI_API_KEY`, `CODEX_API_KEY`, `ANTHROPIC_API_KEY`, `ANTHROPIC_AUTH_TOKEN`, `CLAUDE_CODE_USE_BEDROCK` or `CLAUDE_CODE_USE_VERTEX`. Any of them would silently switch billing to metered API use.
2. The CLI is logged in with a subscription: `codex login status` must report ChatGPT, and `claude auth status` must report `claude.ai`.

A refusal appears in the assistant panel as a `PolicyError`. Unset the variable, or log in again, and retry. The child processes are started with those variables removed from their environment as a second safeguard.

The built-in assistant uses the model `gpt-6.1-sol` at reasoning effort high (the defaults in `src/agentto/llm/cli.py`). Set `AGENTTO_CODEX` or `AGENTTO_CLAUDE` if the CLI is not on your `PATH`.

The assistant is optional. Every design in the paper's 50-brief suite can be built and solved in the workspace alone. The assistant and external agents use the same tools you do.

3

Quick start: a certified release in ten minutes

This chapter takes the motor-mount brief b06 from the task suite to a certified, exported design of 73.7 million elements. You start two processes, read the specification against the brief, climb the resolution ladder, read the certificate and export the STL with its report. No language model is involved.

> **What you need.** A finished installation (Chapter 2) with the solver connected, and a GPU with about 24 GB of memory that no other CUDA program is using. On a smaller GPU, start the backend with a lower ladder cap (Section 3.7); everything else in this chapter stays the same.

3.1 The brief

The brief is a plain-text file, `briefs/b06_motor_mount.md`:

> Plate envelope 160 x 60 x 120 mm (x, y, z), y up, aluminium 6061. It bolts to the machine bed through four M10 bolts on its bottom face (y = 0) at x = 20 and 140 mm, z = 20 and 100 mm; each bolt clamps through a 24 mm washer. Keep a solid boss of 24 mm diameter and 10 mm height around each bolt.
>
> The motor sits on a 40 x 40 mm, 5 mm thick solid pad on the top face, centred at x = 80, z = 60. Its mounting pad carries 2 kN downward and 0.8 kN sideways (+x) from belt tension.
>
> Volume fraction 0.2, 1 mm elements.

3.2 Start the workspace

1. Start the backend and its GPU worker. It listens on `127.0.0.1:8765`.

```
python -B scripts/serve.py
```

2. In a second terminal, start the web front end:

```
node web/node_modules/vite/bin/vite.js web     # or: cd web && npm run dev
```

3. Open `http://127.0.0.1:5173` in Chrome. The **GPU** pill at the bottom right reads *starting* until the worker has loaded CUDA and the solver, then *ready*.

When the backend session is still empty, the workspace opens the suite task `b06_motor_mount` with its reference specification, and shows the brief in the assistant panel. If the session already holds a design (the problem picker then reads **live session**), choose **b06_motor_mount** in the problem picker at the top left.

3.3 Read the specification against the brief

Before solving anything, check that the specification says what the brief says. This is the intent check of Section 1.1, and no software can do it for you. The **Design intent** tree on the left lists every item; click one to see its exact values in the **Inspector** and to highlight it in the viewport. Table 3.1 maps the brief to the specification.

Table 3.1. Brief b06 and its specification. Coordinates are absolute, in millimetres, with the origin at the domain's minimum corner.

Brief	Specification
envelope 160 x 60 x 120 mm, aluminium 6061	domain `[160, 60, 120]`; material $E = 69000$ N/mm^2, $\nu = 0.33$
four M10 bolts through 24 mm washers on the bottom face	supports `bolt_1`–`bolt_4`: elliptical `patch` regions of size 24 x 24 on face `y-`, centred at (x, z) = (20, 20), (20, 100), (140, 20), (140, 100); all directions fixed
solid boss, 24 mm diameter, 10 mm high	keep-in `boss_1`–`boss_4`: cylinders along y, radius 12, span 0–10
40 x 40 mm, 5 mm solid pad on top, centred at x = 80, z = 60	keep-in `pad`: box from (60, 55, 40) to (100, 60, 80)
2 kN down and 0.8 kN in +x on the pad	load `motor`: force `[800, -2000, 0]` N on a 40 x 40 patch of face `y+`
volume fraction 0.2, 1 mm elements	`volume_fraction` 0.2 (of the design volume); resolution $h = 1$

The **Checks** panel should read *All rules pass*, and the top bar *valid spec*. The ladder at the bottom then shows four buttons.

3.4 Climb the ladder

The ladder for this specification has four named rungs. The spec's 1 mm elements are the base mesh; the release rung is the finest grid that fits on the GPU, here 0.25 mm:

Rung	Element edge	Grid	Elements
Sketch	2 mm	80 x 30 x 60	144,000
Preview	1 mm	160 x 60 x 120	1.15 million
Refine	0.5 mm	320 x 120 x 240	9.2 million
Release	0.25 mm	640 x 240 x 480	73.7 million

1. Click **Sketch**. Density frames stream into the viewport while it runs. The first sketch of a new specification builds its multigrid hierarchy and takes a few seconds; on the paper's workstation the cold b06 sketch took 3.4 s.

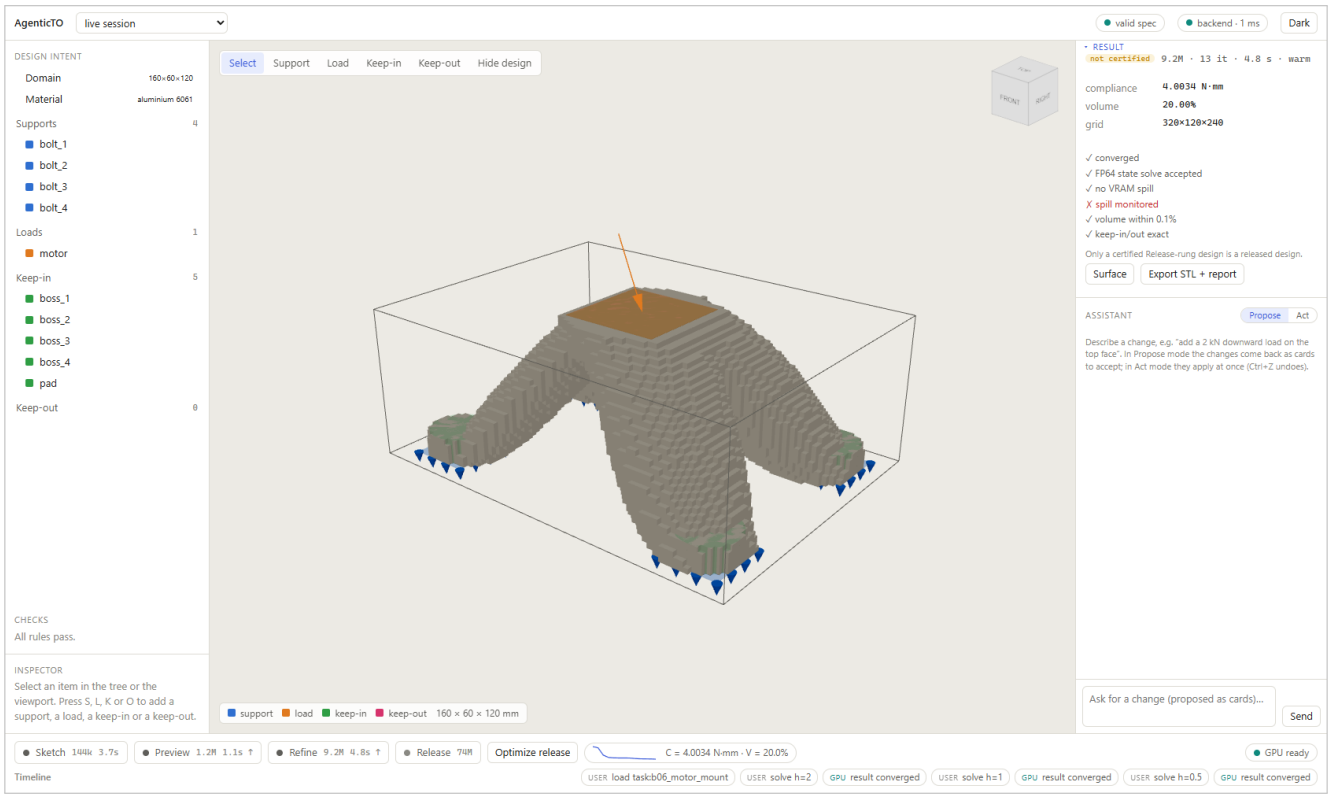


Figure 3.1. The motor mount after Refine (9.2 million elements). The result card at the top right shows the checks: five pass, but *spill monitored* fails because spill monitoring runs only on the release rung, so the card says *not certified*. That is expected for every rung below Release. Screenshot from `scripts/gui_screenshots.py`.

2. Click **Preview**, then **Refine**. Each starts from the design of the rung below (marked ↑ on the button) and takes seconds. Figure 3.1 shows the refine design.

3. When Refine has converged, click **Optimize release**. It re-opens the optimization on the 73.7-million-element grid for about 60 iterations. On the paper's workstation this policy used 61% of the compute of a cold release-rung run (423 s) and ended within 0.03% of its compliance [C3.1]. Alternatively, **Release** certifies the upsampled refine design in about 12 iterations (18 s of compute for b06 in the latency study [C6.4]), but it barely changes the design (Section 7.3).

3.5 Read the certificate

When the release finishes, the result card shows the latest design and its certificate. A release is finished only when the card reads **certified**: all six checks carry a ✓, and the grid is the release grid (here 640 x 240 x 480). Chapter 7 explains each check. The certified design is shown as its $\rho = 0.5$ surface; **Voxels** switches to the raw density field.

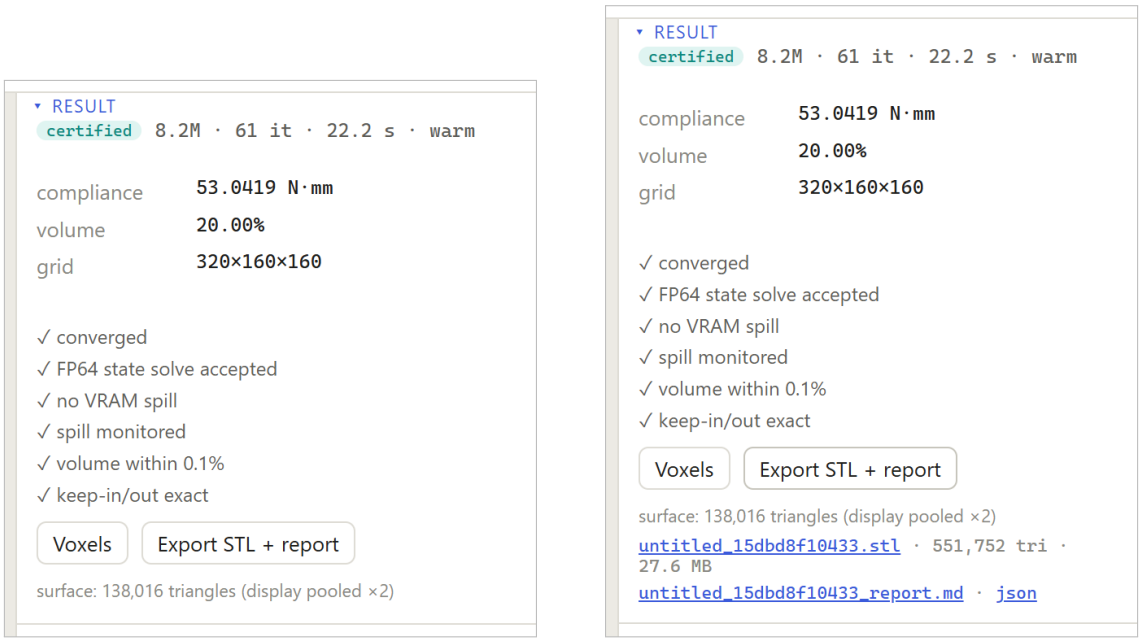


Figure 3.2. Left: the result card of a certified release, with all six checks passing. Right: the card after export, with links to the STL and the two report files. These two screenshots come from the bracket of Chapter 5 on a reduced ladder; the card for b06 has the same layout.

3.6 Export

Click **Export STL + report**. Three files are written to the session's `exports/` folder, under `sessions/`⟨*date-time*⟩`/exports/`, and linked from the card (Figure 3.2):

- `b06_motor_mount_`⟨*job*⟩`.stl`: a closed binary STL of the full-resolution $\rho = 0.5$ surface (in the latency study the export at release took 0.3–2.2 s [C6.4]);
- `b06_motor_mount_`⟨*job*⟩`_report.md` and `_report.json`: the specification and its hash, the rung, grid and solver, iterations and compute time, the compliance, every certificate check, the STL's hash, and the hash of the session log at the moment of export (Appendix C).

A design that is not a certified release is exported with `_draft` in its file name, so a draft cannot be mistaken for a release later.

> **You are done.** You now hold a certified design, its STL, a report that ties it to the exact specification and solver settings, and a session log in `sessions/`⟨*date-time*⟩`/session.jsonl` from which the whole session can be replayed (Chapter 10).

3.7 On a smaller GPU

Cap the ladder when you start the backend:

```
python -B scripts/serve.py --ladder-max 1.2e7
```

The cap can lower the default of 8×10^7 elements but never raise it. With this cap b06 gets the rungs 18,000 / 144,000 / 1.15 million / 9.2 million, and the 9.2-million-element rung becomes its release rung. The certificate means the same on any rung that is the release rung of its ladder: the result is certified on the grid it reports, and the report states that grid.

3.8 The same task with an agent

The paper ran this task without a person: a Codex agent connected over MCP built the specification from the brief, asked four clarification questions (answered by a scripted oracle holding the reference reading), climbed the ladder and exported a certified 73.7-million-element release in 176 s from its first tool call. Its specification equalled the reference reading in every field, and the log replayed exactly. Chapter 8 shows how to do the same with the built-in assistant, and Chapter 9 how to connect an agent of your own.

4

The workspace

The workspace is a single browser page with ten areas. This chapter names each area, explains the status indicators, and shows how to move around the 3D view. Later chapters use these names.

4.1 The areas

Figure 4.1 and Table 4.1 name the ten areas. Three of them carry most of the work: the **Design intent** tree with its **Inspector** below (where you state the problem), the viewport (where you place regions by clicking faces), and the ladder with the result card (where you solve and read the outcome).

4.2 Status indicators

Specification status (top bar). *valid spec*, *valid · n warnings*, or *n errors*. It counts rule findings and schema errors. The ladder is hidden or disabled while there is an error.

Backend (top bar). Whether the WebSocket to the backend is open, with the round-trip time in milliseconds.

GPU (bottom right). *off* before the first solve request, *starting* while the worker process loads CUDA and the solver, *ready*, or *dead* if the worker exited (Chapter 12).

Rung buttons . Each shows its element count and, after a solve, its compute time; an ↑ marks a warm start from the rung below, a seal marks a certified result. While a rung runs it shows *setting up…* and then the iteration count.

Result card badges . **certified** or **not certified**, and **stale** when the specification has changed since this design was solved (Section 7.5).

4.3 First launch

On first launch a **Getting started** card sits above the result card (Figure 4.2). It summarizes the workflow in three steps and the two shortcuts worth learning first: ? for the keyboard map and Ctrl + K for the command palette. **Got it** hides it.

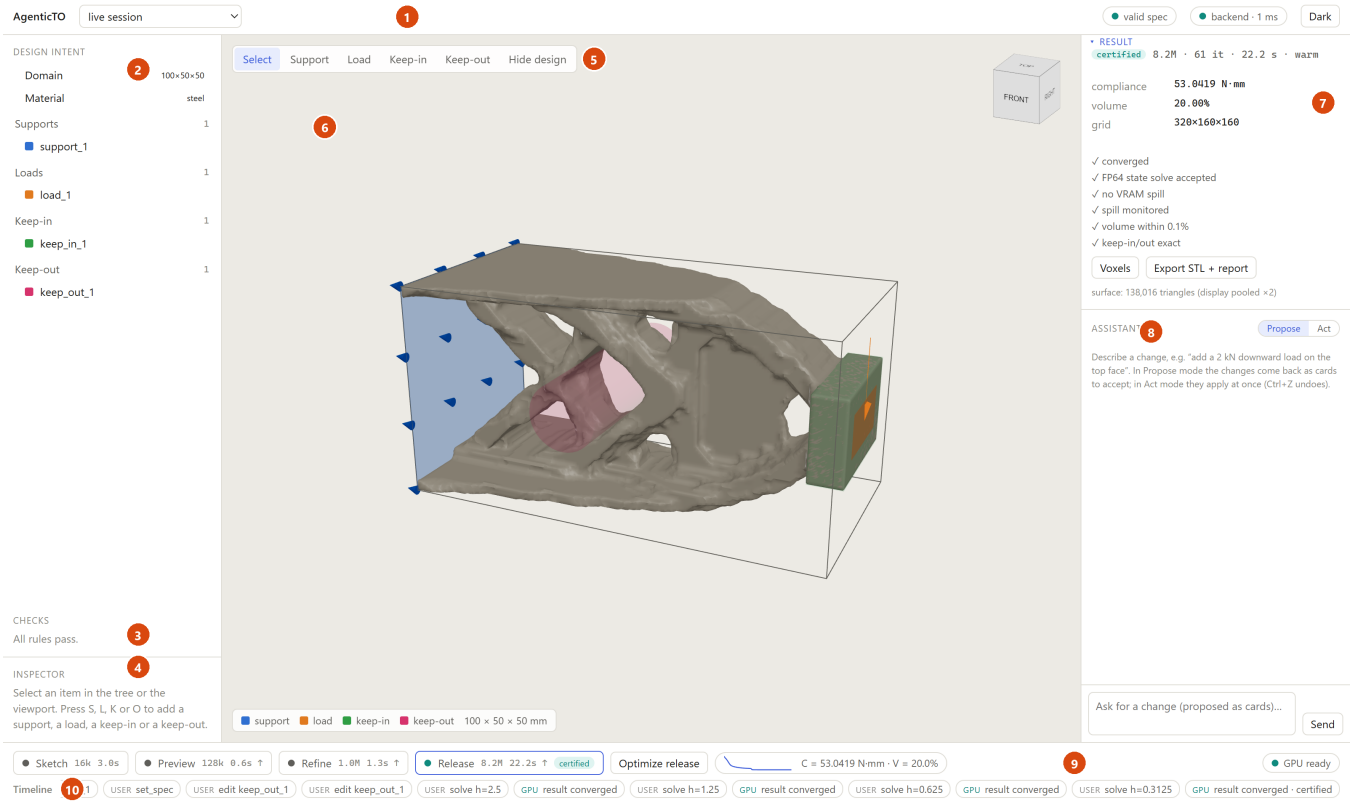


Figure 4.1. The workspace after a certified release of the bracket built in Chapter 5. The numbers refer to Table 4.1.

4.4 Moving around the 3D view

- Left-drag orbits, right-drag pans, the mouse wheel zooms.
- Click a face of the orientation cube to look along that axis.
- Click an item in the viewport or in the tree to select it; the inspector shows its values.
- **Hide design** hides the density so you can see the regions inside it.

4.5 The timeline

The timeline at the bottom is the session log made visible (Figure 4.3). Each record is a tool call or a solve result, tagged with its actor. Agent edits appear here just like yours. Chapter 10 describes the log behind it.

4.6 Themes, keyboard and palette

Dark/**Light** in the top bar switches the theme (Figure 4.4). ? opens the keyboard map and Ctrl + K the command palette, which runs any rung, **Optimize release**, **Stop**, shows the last surface, exports, undoes and redoes, picks a tool and switches the theme. Appendix D lists all shortcuts. Shortcuts are off while you type in a field.

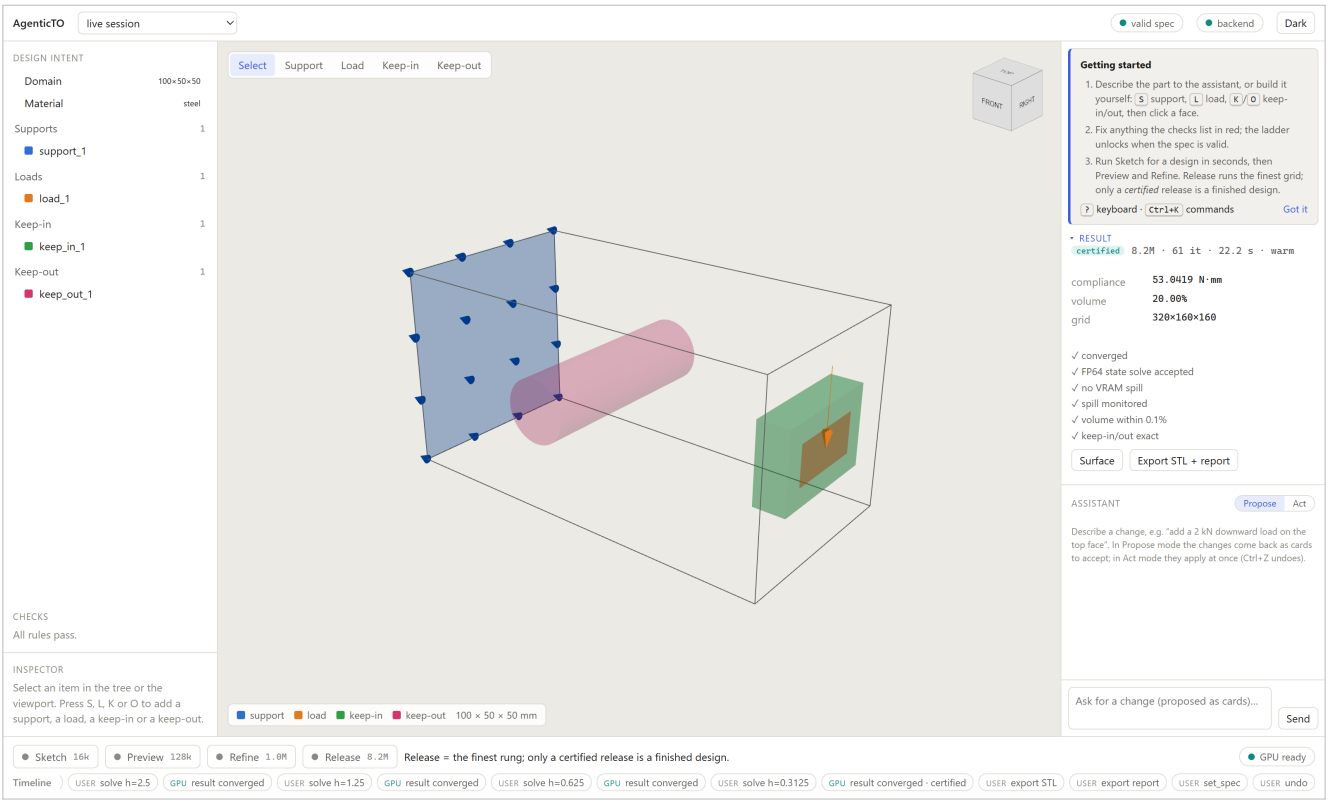


Figure 4.2. The **Getting started** card at the top right. In this screenshot the backend session already held the bracket of Chapter 5, so the problem picker reads **live session**; on an empty session the workspace opens the suite task b06 instead.

Figure 4.3. The timeline: user edits, solves and GPU results, each tagged with its actor.

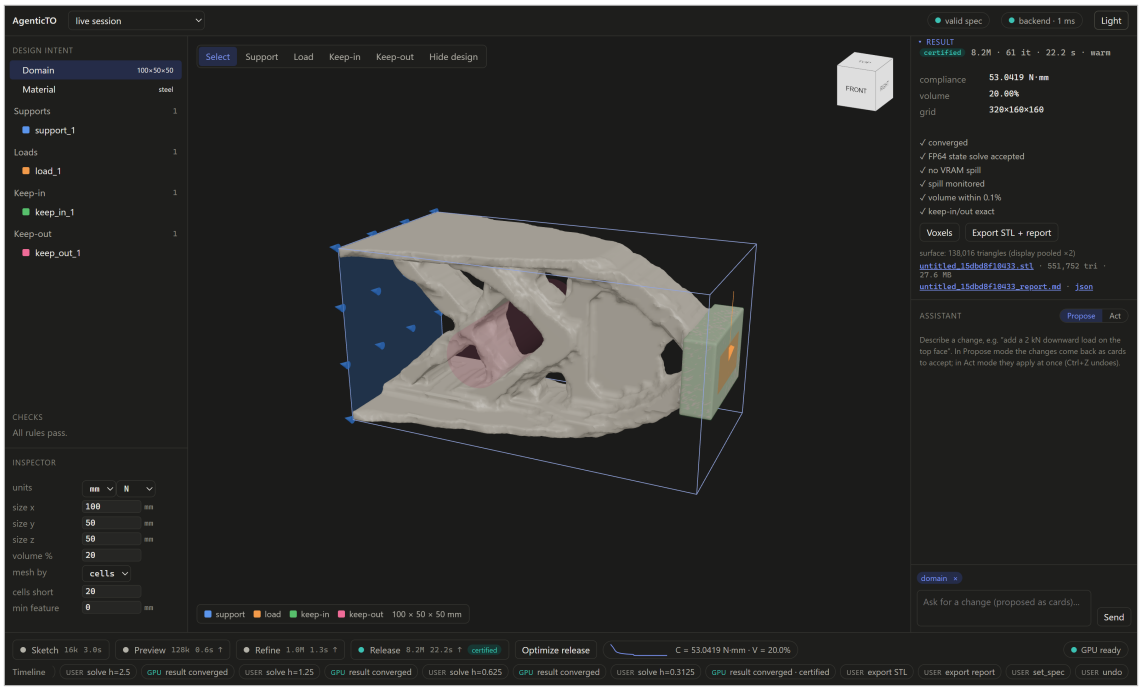


Figure 4.4. The dark theme.

Table 4.1. The ten areas of the workspace (Figure 4.1).

#	Area	What it does
1	**Top bar**	Problem picker (**New design…**, the 50 suite tasks, the three template problems), specification status, backend connection and round-trip time, light/dark theme.
2	**Design intent**	The specification as a tree: domain, material, supports, loads, keep-in and keep-out regions. Click an item to select it; a red **error** or amber **check** badge marks an item that a rule flags.
3	**Checks**	Live rule findings, each with its code, severity and message.
4	**Inspector**	Exact values of the selected item, edited in place: region kind, face, centre, size, force, fixed directions.
5	**Viewport tools**	**Select**, **Support**, **Load**, **Keep-in**, **Keep-out**; **Hide design**.
6	**3D viewport**	The domain with supports (blue), loads (orange arrows), keep-in (green) and keep-out (pink) regions; the density design or its surface; the orientation cube; a size legend.
7	**Result card**	The latest finished design: certified or not, current or stale, compliance, volume, grid, each certificate check, **Surface**/**Voxels**, **Export STL + report**.
8	**Assistant**	Chat with the design assistant, the **Propose**/**Act** dial, clarification questions and proposed changes as cards.
9	**Resolution ladder**	**Sketch**, **Preview**, **Refine** and **Release** with their element counts, times and certificate seals; **Optimize release**; **Stop**; live compliance and volume with a convergence sparkline; the **GPU** status pill.
10	**Timeline**	The session log, newest on the right, each record tagged **USER**, **AGENT**, **GPU** or **SYSTEM**. Hover a record for its time and details.

5

Building a specification

The specification is the one document that says what problem is solved. This chapter builds one by hand in the workspace, a small steel bracket with a solid pad and a bolt hole, and then explains each part of the document: the coordinate frame, units, material, supports, loads, keep-in and keep-out regions, the volume fraction and the mesh.

5.1 Tutorial: a cantilever bracket

The bracket is a 100 x 50 x 50 mm steel block, fixed at one end and loaded at the other, with a mounting pad that must stay solid and a hole that must stay empty. The screenshots were recorded with the ladder capped at 1.2×10^7 elements, so that the release solves in about a minute.

5.1.1 Start a new design

Choose **New design…** in the problem picker. You get an empty 100 x 50 x 50 mm box in steel ($E = 210000$, $\nu = 0.3$, in N and mm) with a volume fraction of 0.3 and 20 elements along the shortest edge. The **Checks** panel says what is missing, and the ladder stays hidden until the specification is valid (Figure 5.1).

Select **Domain** in the tree to set the size, the units, the volume fraction (**volume %**), the mesh (by element size h or by cells along the shortest edge) and an optional minimum feature size. Select **Material** to set its name, Young's modulus E and Poisson's ratio ν.

5.1.2 Add a support

Press S (or click **Support**) to arm the support tool. The toolbar asks you to click a face of the domain (Figure 5.2). Click the face at $x = 0$. It becomes a support fixed in all three directions, the tool returns to **Select**, and the new support is selected in the inspector. There you can change the region kind (face, patch, edge, box or cylinder) and untick fixed directions to make a roller.

5.1.3 Add a load

Press L and click the face at $x = 100$. A 1 kN load appears on a small patch centred where you clicked, pointing into the face. Edit it in the inspector: here the force becomes 2000 N downward (force $y = -2000$) and the patch 10 x 20 mm (Figure 5.3). Each edit takes effect as you type and can be undone.

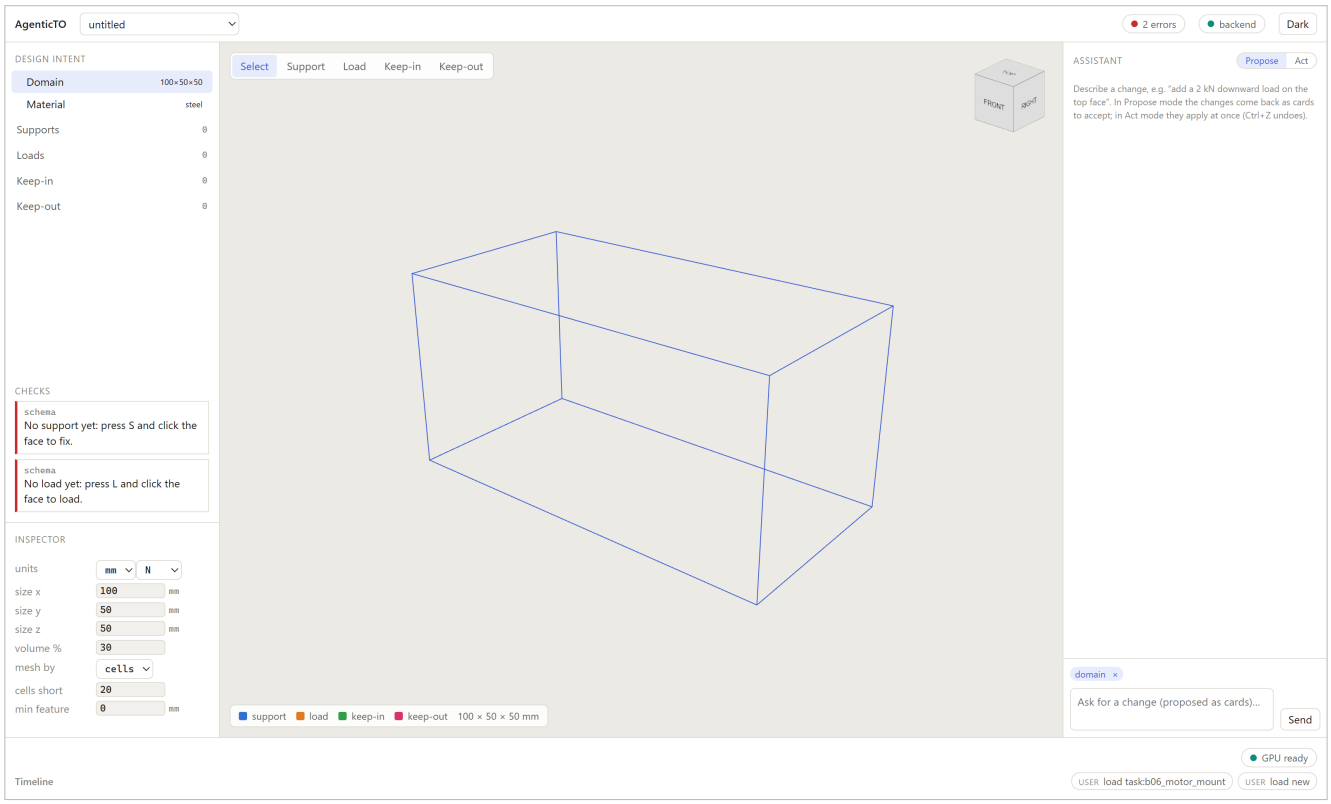


Figure 5.1. A new design. The checks say what is missing and how to add it.

5.1.4 When a check fails

The rules run 300 ms after every edit. Untick y and z on the support, so that it fixes only x: rule **R003** reports the rigid-body motions left free (translation in y and z, rotation about x), the support gets an **error** badge in the tree, and the ladder locks (Figure 5.4). Tick y and z again and the error clears. Errors block solving; warnings and notes do not.

5.1.5 Keep-in and keep-out regions

Press K and click a face to add a keep-in region (always solid), or O for a keep-out region (always empty). Each starts as a slab 10% thick against that face. In the inspector a keep-in or keep-out region can be a box or a cylinder, and you can switch it between keep-in and keep-out.

In Figure 5.5 the keep-in region was resized to an 8 x 20 x 30 mm pad under the load, using its min/max fields. The keep-out region was converted to a cylinder of radius 8 mm along z, centred at $x = 40$, which leaves a through-hole for a bolt or a cable.

5.1.6 Undo and redo

Ctrl+Z undoes and Ctrl+Shift+Z or Ctrl+Y redoes, across your edits and agent edits alike. Every step is a record in the timeline. Chapter 7 continues with this bracket and solves it.

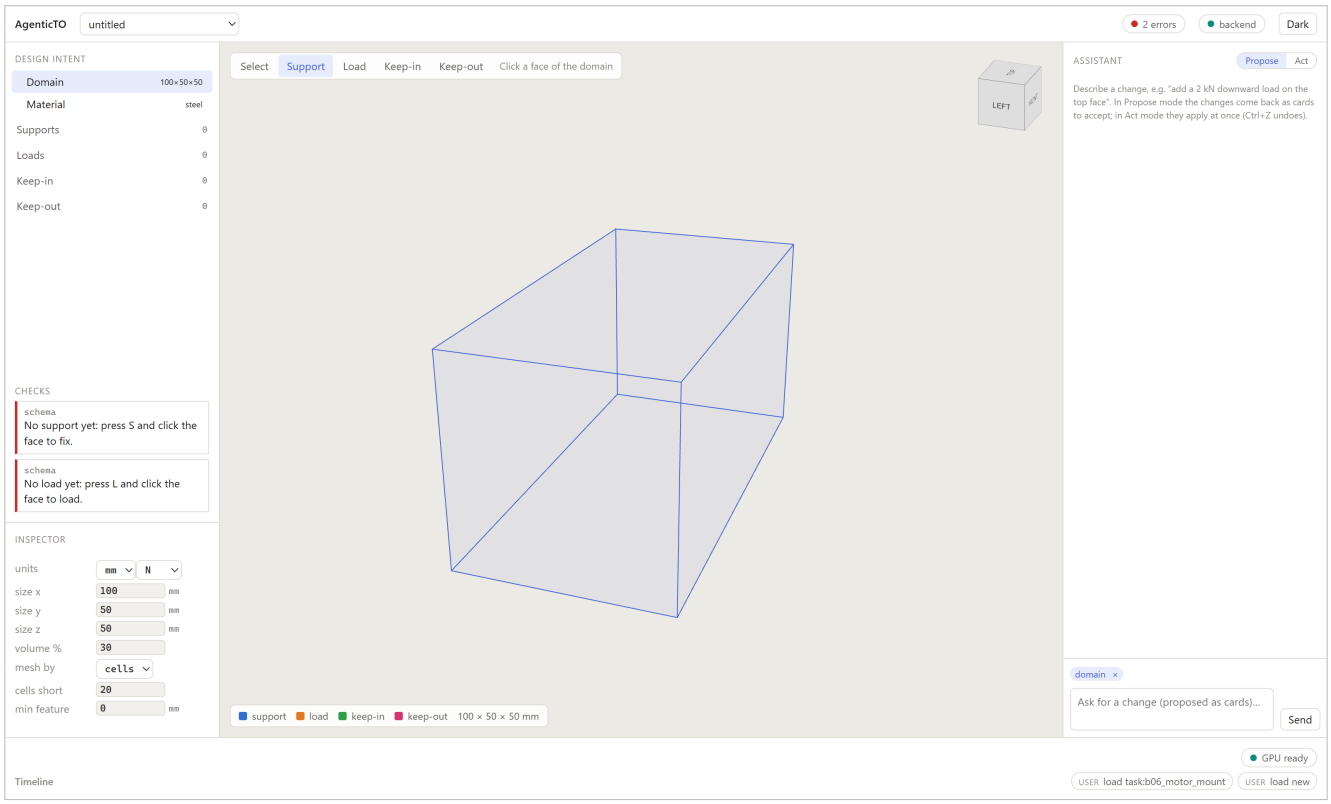


Figure 5.2. The support tool armed: click the face to fix.

5.2 The document

Whatever you do in the workspace ends up as one JSON document. This is the reference specification of suite task b01:

```
{
  "schema_version": "agentto.spec/v0",
  "name": "b01 cantilever block",
  "units": {"length": "mm", "force": "N"},
  "domain": {"size": [200, 100, 100]},
  "material": {"name": "steel", "E": 210000.0, "nu": 0.3},
  "volume_fraction": 0.3,
  "supports": [{"id": "wall", "region": {"kind": "face", "face": "x-"}}],
  "loads": [{"id": "pad", "force": [0, -1000, 0],
             "region": {"kind": "patch", "face": "x+", "center": [50, 50],
                        "size": [10, 10], "shape": "rect"}}],
  "passive": [],
  "resolution": {"h": 1.0}
}
```

The schema is strict: unknown fields are rejected, every `id` must be unique across supports, loads and passive regions, and a document needs at least one support, one load, a volume fraction and a resolution. A draft that is not complete yet (no load, say) is kept as a draft; the tools report what is missing and the rules do not run until it is complete. Appendix A lists every field.

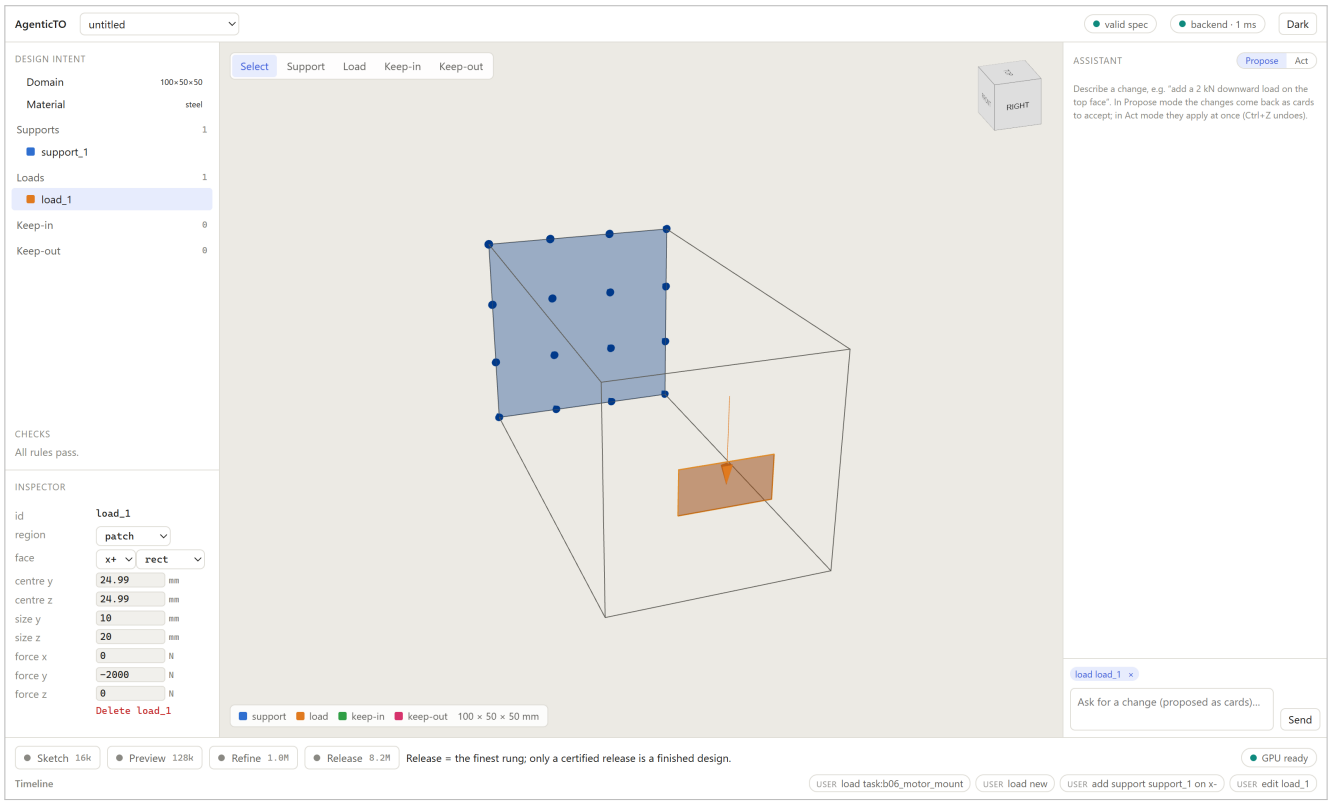


Figure 5.3. A load placed by clicking and then edited in the inspector.

5.2.1 Coordinate frame

The domain is an axis-aligned box from $(0, 0, 0)$ to the size you give. All coordinates are absolute, in that frame. The faces are named `x-`, `x+`, `y-`, `y+`, `z-` and `z+`: `x-` is the face at $x = 0$, `x+` the face at $x = L_x$. Which axis is "up" is yours to choose; the brief b06, for example, says "y up".

A patch on a face gives its centre and full size in the face's two in-plane axes, always in increasing axis order: (y, z) on x faces, (x, z) on y faces, (x, y) on z faces. This is the most common source of misplaced regions. `list_regions` prints the in-plane axes of every face, and the viewport shows the result.

5.2.2 Units

Lengths are in mm, m or in; forces in N, kN or lbf. The units label the numbers; they do not convert them.

> **Units relabel; they do not convert.** Switching from mm to m changes the label on the numbers, not the numbers. Enter every value in the units shown, and keep E consistent with them: in N and mm, E is in N/mm^2 (MPa), so steel is 210000. In kN and mm, steel is 210. In the paper's second-provider benchmark, two of the four remaining wrong answers were exactly this slip: the agent worked in kN and mm but left E in MPa, a material 1000 times too stiff.

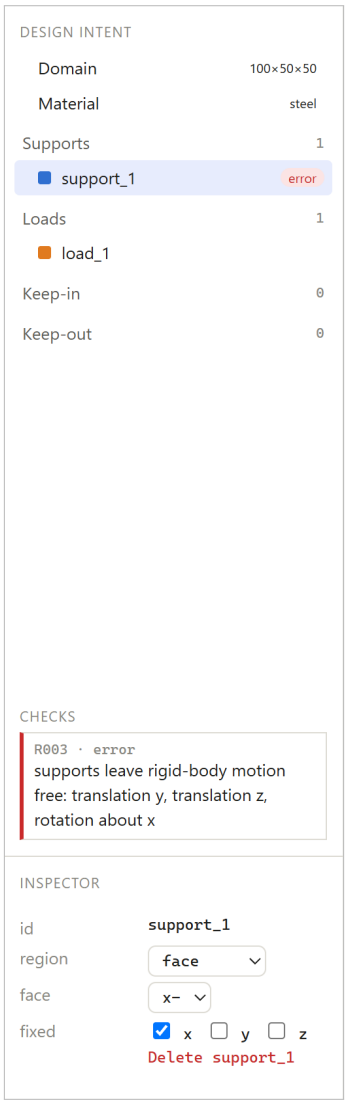


Figure 5.4. Rule R003: the support leaves rigid-body motion free. The ladder stays locked until the support is fixed.

5.2.3 Material

An isotropic linear-elastic material: a name, E (positive) and ν (between -1 and 0.5). The name is a label; only E and ν enter the computation. Because the objective is compliance under one load case, the optimal layout does not depend on E, but the reported compliance does, and the specification must state the problem the engineer meant.

5.2.4 Regions

Supports and loads select grid *nodes*; keep-in and keep-out regions select *elements* by their centres. Table 5.1 lists the five region kinds.

Selection uses a rounding tolerance of 10^{-9} element edges. A patch with no node within half its size of its centre, or an edge between grid lines, is snapped to the nearest grid line and reported by rule **R010**; nothing is snapped silently. A region that selects nothing is an error (**R002** for a support, **R004** for a load) or a warning (**R011** for a passive region).

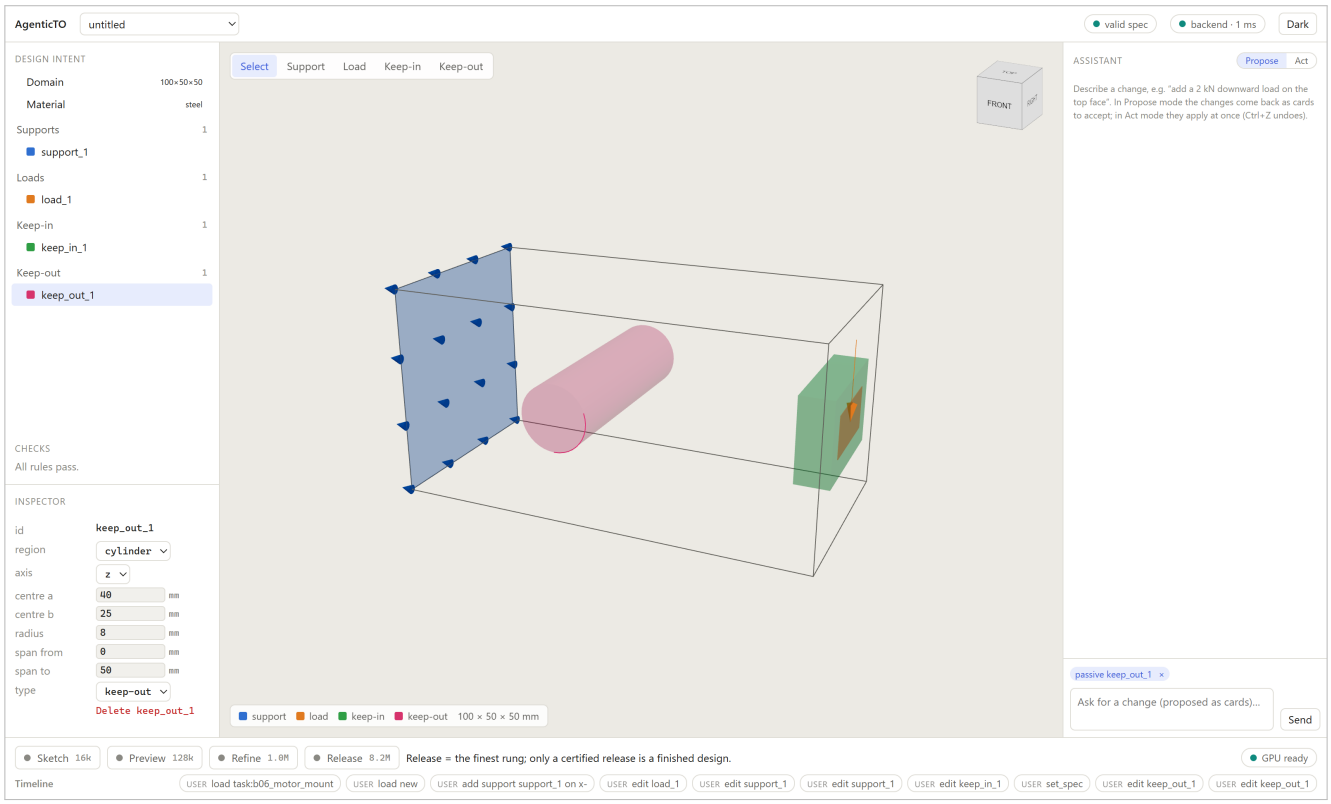


Figure 5.5. The finished setup: support (blue), load on a solid pad (green) and a keep-out cylinder (pink). All rules pass.

Table 5.1. Region kinds. All lengths are in the spec's length unit; "in-plane order" is the increasing axis order of the face's two in-plane axes.

Kind	Fields	Valid for
`face`	`face`: one of `x-`, `x+`, `y-`, `y+`, `z-`, `z+`	supports, loads
`patch`	`face`; `center` and `size` in in-plane order; `shape` `rect` (default) or `ellipse` (the inscribed ellipse, e.g. a washer)	supports, loads
`edge`	`axis`; `at`: the other two coordinates in increasing order; optional span [start, end] along the axis	supports, loads
`box`	`min`, `max`: opposite corners, `min` < `max` on every axis	supports, loads, keep-in/out
`cylinder`	`axis`; `center`: the axis position in the other two coordinates; `radius`; span [start, end] along the axis	supports, loads, keep-in/out

5.2.5 Supports

A support fixes the nodes of its region in the directions listed in `fix`; the default is all three. A roller fixes fewer: `"fix": ["y"]` on a bottom edge leaves the span and width directions free. Supports only need to remove the six rigid-body modes together; rule **R003** checks exactly that.

5.2.6 Loads

A load is a total force vector, in the spec's force unit, shared equally by the nodes of its region. This keeps the total force the same on every mesh, but it is not a pressure integration: on a

curved or partial region the nodal shares are equal, not area-weighted. All loads act together as one load case. A load component that falls on a fixed degree of freedom is dropped, because it would go straight into the support, and rule **R005** reports the share.

5.2.7 Keep-in and keep-out regions

In the document these are `passive` items with `kind` `solid` (keep-in) or `void` (keep-out), each a box or a cylinder. An element whose centre falls inside is held at density 1 or 0 at every iteration and is not a design variable. Keep-out elements keep a small stiffness rather than zero, so their nodes stay well defined. The certificate checks that every keep-in element ends at exactly 1 and every keep-out element at exactly 0.

5.2.8 Volume fraction

The volume fraction v^* is the share of the *design* volume, which is the domain minus every keep-in and keep-out element:

$$\frac{1}{|\mathcal{D}|}\sum_{e\in\mathcal{D}}\bar{\rho}_e \le v^*, \qquad \bar{\rho}_e = 1 \ (e\in\mathcal{S}), \quad \bar{\rho}_e = 0 \ (e\in\mathcal{V}), \tag{5.1}$$

where $\mathcal{D}$ are the design elements, $\mathcal{S}$ the keep-in and $\mathcal{V}$ the keep-out elements, and $\bar{\rho}$ the filtered and projected density. Adding a keep-in pad therefore adds material on top of the budget; it changes both the feasible set and the total material. That is why an agent that invents a pad the brief did not ask for has changed the problem, even if the result looks stiffer.

5.2.9 Resolution and the minimum feature

Give the mesh as an element edge `h` or as `cells_short_edge`, the number of elements along the shortest side; exactly one of the two. The edge must divide all three sides into whole numbers of elements, or rule **R001** fires. This resolution is the *base* mesh from which the ladder is built. The release rung is the finest rung that fits on the GPU and is often finer: for b06 the spec says 1 mm and the release rung is 0.25 mm (Section 7.1).

The optional `min_feature` is the smallest member you need. It does not constrain the optimizer; it only feeds rule **R009**, which warns when that size spans fewer than about 2.4 elements and would be filtered out.

5.3 Editing through the tools

Every edit in the workspace is a call to one of the editing tools, logged like an agent's call. The same calls work from Python and over MCP:

- `new_spec` starts a box domain; `set_spec` replaces the whole document.
- `set_params` sets scalar fields (name, size, units, E, ν, material name, volume fraction, h or `cells_short_edge`, `min_feature`); only the arguments given change.

- `add_support`, `add_load` and `add_passive` add items; `update_item` changes fields of one; `remove_item` deletes one.
- `undo` and `redo` step through the history.

A failed edit leaves the specification unchanged. Every successful edit is undoable and records the hash of the resulting specification in the log, so any later result can be tied to the exact document it was solved from.

> **Ground before you solve.** `list_regions` reports, for every item, its physical bounding box and how many nodes or elements it selects at a given mesh, with any snapping. `explain_spec` prints the whole specification in plain sentences, and `render` draws the supports, loads and passive regions as a picture. All three are cheap; use them before the first solve.

6

Rules

Thirteen deterministic rules check that a specification states an admissible problem before anything is solved. This chapter explains when the rules run, how to read a finding, and what each code means and how to fix it. The rule list in this chapter is generated from `src/agentto/rules.py`.

6.1 When the rules run

- In the workspace, 300 ms after the last edit. In the paper's latency study the 95th-percentile validation time per task was 1.2–10.5 ms [C6.4].
- On every `validate_spec` call, at the specification's own mesh or at a mesh size `h` you pass.
- On every `solve` call, again at the mesh of the requested rung. A solve whose rung fails validation is refused before it reaches the GPU, and the GPU worker checks once more before it starts.

Any error blocks solving and release. A warning or info finding does not block; it stays visible in the **Checks** panel, often with a suggested fix. Several rules depend on the grid (for example R002, R010, R011 and R013), so a specification can pass at one mesh and fail at another.

6.2 Reading a finding

Each finding has a stable code, a severity, a message, the ids of the items it concerns (the tree marks those items), and, where one exists, a fix. A fix either sets a field (the workspace can offer it as one click, and an agent can propose it as a card) or suggests an action in words. `validate_spec` returns the findings as JSON:

```
{"ok": false,
 "resolution": {"h": 1.0, "grid": [100, 50, 50], "n_elem": 250000, ...},
 "findings": [{"code": "R003", "severity": "error",
   "message": "supports leave rigid-body motion free: translation y, translation z,
    ↪ rotation about x",
   "ids": ["support_1"],
   "fix": {"suggest": "add a support that restrains translation y, translation z,
    ↪ rotation about x"}}]}
```

The codes are stable: experiments, logs and this book refer to them, and a code is never reused for a different check.

6.3 Summary

Code	Severity	Fires when
R001	error	no grid fits the domain at this element size
R002	error	a support selects no nodes
R003	error	the supports leave a rigid-body mode free
R004	error	a load selects no nodes, or every load is zero
R005	error / warning	part of the load acts on fixed degrees of freedom
R006	error	no design elements remain
R007	error	elements are both keep-in and keep-out
R008	error	a support or load lies entirely inside a keep-out region
R009	warning	the minimum feature is not resolvable at this mesh
R010	warning	a region was snapped to the grid
R011	warning	a passive region contains no element centre
R012	warning / info	supports change on the coarsest multigrid level
R013	error	the grid needs more GPU memory than is available

6.4 Rule by rule

In the messages below, ⟨*id*⟩ and similar placeholders stand for values filled in at run time.

R001 error no grid fits the domain at this element size

Message `domain ⟨axis⟩ size ⟨length⟩ is not a whole number of elements of edge ⟨h⟩`

Meaning A side of the domain is not a whole number of elements of edge h (or the number of cells along the short edge gives a non-integer count on another side). R001 stops the check: no other rule runs until a grid exists.

Fix Choose an h that divides all three sides, or accept the one-click fix, which sets `cells_short_edge` to a value that fits. *(The finding carries a fix: one click: set* `resolution.cells_short_edge`*.)*

R002 error a support selects no nodes

Message `support '⟨id⟩' selects no nodes`

Meaning The support's region misses every grid node: a box between grid planes, a cylinder thinner than the node spacing, an edge span outside the domain. The support does nothing.

Fix Run `list_regions` to see the node count of each item, then move or enlarge the region, or refine the mesh.

R003 error the supports leave a rigid-body mode free

Message `supports leave rigid-body motion free: ⟨modes⟩`

Meaning The fixed degrees of freedom do not remove all six rigid-body modes, so the stiffness matrix is singular. The message names the free translations and rotations. A common cause is a single edge support (free rotation about the edge) or rollers that fix only one direction.

Fix Add a support, or fix more directions, so that the listed modes are restrained. Fix in the spec; do not rely on the solver to fail. *(The finding carries a fix: suggests: add a support that restrains modes.)*

R004 error a load selects no nodes, or every load is zero

Message `every load has zero force` `load '⟨id⟩' selects no nodes`
Meaning Either a load region misses every node, or the force vectors of all loads are zero, so there is nothing to optimize against.
Fix Check the load's region with `list_regions` and its force components with `explain_spec`.

R005 error / warning part of the load acts on fixed degrees of freedom

Message `⟨share⟩% of the applied load acts on fixed DOFs and is lost`
Meaning The problem builder drops every load component that falls on a fixed degree of freedom (it would go straight into the support) and records the share. Above 50% of the load the finding is an error, otherwise a warning.
Fix Move the load off the support, or leave that direction free at the shared nodes.

R006 error no design elements remain

Message `no design elements remain after passive regions`
Meaning The keep-in and keep-out regions together cover every element, so there is nothing to design.
Fix Shrink or remove passive regions.

R007 error elements are both keep-in and keep-out

Message `⟨n⟩ elements are both keep-in and keep-out`
Meaning A keep-in and a keep-out region overlap. An element cannot be held solid and empty at once.
Fix Remove the overlap: shrink one region, or split it.

R008 error a support or load lies entirely inside a keep-out region

Message `⟨support|load⟩ '⟨id⟩' lies entirely inside a keep-out region`
Meaning Every element around the item's nodes is held empty, so the load has no material path (or the support holds nothing). The solver can still converge on such a problem because empty elements keep a small stiffness; in the paper's gate ablation it converged and passed the numerical certificate.
Fix Move the item out of the keep-out region, or move the keep-out region.

R009 warning the minimum feature is not resolvable at this mesh

Message `min feature ⟨min feature⟩ is ⟨n⟩ elements; members below about ⟨2.4⟩ elements are filtered out`
Meaning `min_feature` spans fewer than about 2.4 elements (twice the final filter radius of 1.2 elements); members that thin are filtered out.

Fix	Accept the one-click fix, which sets h to `min_feature`/2.4, or relax `min_feature`. *(The finding carries a fix: one click: set* `resolution.h`*.)*

R010 warning a region was snapped to the grid

Message	⟨*n*⟩
Meaning	A patch had no node within half its size of its centre, or an edge fell between grid lines; it was moved to the nearest grid line. Nothing is snapped silently.
Fix	Usually harmless. If the change matters, align the coordinates with the grid or refine the mesh.

R011 warning a passive region contains no element centre

Message	`passive region '`⟨*id*⟩`' contains no element centre at h=`⟨*h*⟩
Meaning	At this element size no element centre falls inside the keep-in or keep-out region, so it has no effect.
Fix	Refine the mesh or enlarge the region. *(The finding carries a fix: suggests: refine the mesh or enlarge the region.)*

R012 warning / info supports change on the coarsest multigrid level

Message	`on the coarsest multigrid level (1/`⟨*stride*⟩ `resolution) the supports leave rigid modes free (`⟨*modes*⟩`); the solver switches to the dilated coarse support rule (RobustGMG)` ⟨*ids*⟩ `are not represented on the coarsest multigrid level (1/`⟨*stride*⟩`); the solve stays correct but may need more iterations`
Meaning	On the coarsest multigrid level the supports either leave rigid modes free (warning: the solver switches to the dilated coarse support rule) or lose some supports (info: the solve stays correct but may need more iterations). Neither blocks solving.
Fix	No action is needed. The finding explains a slower solve.

R013 error the grid needs more GPU memory than is available

Message	⟨*g.n_elem / 1000000.0*⟩`M elements need about `⟨*need / 1000000000.0*⟩ `GB with `⟨*solver*⟩`; the GPU has `⟨*vram_bytes / 1000000000.0*⟩ `GB`
Meaning	The memory estimate (bytes per element times elements) exceeds the GPU memory. The estimate uses about 366 bytes per element for `mg64` and 146 for `mg_ir`.
Fix	Solve a coarser rung. A solve checks with `mg_ir`; validation at the spec's own mesh checks with `mg64`. *(The finding carries a fix: suggests: use mg_ir or a coarser rung.)*

6.5 What the rules cannot see

The rules decide whether a problem is admissible, not whether it is the problem the engineer meant. A support that selects real nodes and removes every rigid-body mode passes, even if it restrains a direction the brief leaves free. A keep-in pad that the brief never mentioned passes. A load with the right magnitude on the wrong face passes. These are intent errors, and only the engineer, helped by clarification, can catch them (Chapter 8).

> **From the paper.** With grounding tools but no questions, the primary agent produced 6 wrong specifications on the 50-brief suite. All 6 passed every rule [C4.2]. Conversely, the rules stopped all 3 deliberately ill-posed briefs before any solve: two rigid-body cases (**R003**) and one load inside its own keep-out region (**R008**) [C4.1].

The rules also do not check that the final design is connected, that the exported surface is manufacturable, or that the structure is safe. The release certificate (Chapter 7) covers the numerical computation, and nothing in AgenticTO covers structural safety.

7

The resolution ladder and the release certificate

A specification is solved on a ladder of grids, each twice as fine per axis as the one below. This chapter explains how the rungs are chosen, how one rung starts from the design of the one below, the three warm release policies and what each can and cannot do, and the six predicates of the release certificate.

7.1 The rungs

The ladder is built from the specification's base mesh h. Candidate element sizes are $h \cdot 2^k$ for integer k; a candidate is kept when its grid divides the domain into whole elements and has between 5000 and 8×10^7 elements. Consecutive rungs must differ by exactly a factor of two per axis, so that every rung can start from the one below. The four finest rungs are named *sketch*, *preview*, *refine* and *release*; any coarser ones are named `coarse0`, `coarse1`, … and are not shown in the workspace.



Figure 7.1. The ladder of the motor mount b06: element counts per rung. Each rung has eight times the elements of the one below and starts from its design. Only the release rung runs with spill monitoring, so only it can be certified.

The release rung is the finest grid on the ladder, not the specification's h. When a brief asks for elements of at most 1 mm, a release at 0.25 mm satisfies it; `ladder` says so in a note. Asking `solve` for a rung and an `h` that disagree is an error, so an agent cannot accidentally solve the refine grid under the release name.

Solver choice. Grids up to 6×10^7 elements use the FP64 multigrid solver `mg64`; larger grids use `mg_ir`, a mixed-precision iterative-refinement variant that needs less memory. If `mg_ir` fails on a grid of at most 6.6×10^7 elements, the worker retries once on `mg64`.

Typical times. Table 7.1 gives the latency study's numbers on the RTX 4090.

Table 7.1. Backend timings over the 14 released tasks of the session study, each measured through the running backend [C6.4, C6.5]. Warm means started from the design of the rung below or, for the sketch, from the previous sketch after an edit.

Interaction	Range over tasks
Edit → validation, 95th percentile	1.2–10.5 ms
Edit → warm sketch, 95th percentile	0.59–2.40 s
Cold sketch of a new specification	2.5–6.7 s (4 of 14 within 3 s); first density frame after about 0.1 s
Release, warm (tail policy), compute	3.1–56.4 s, all 14 certified

7.2 Warm starts

When a solve has a suitable earlier design, it starts from it:

1. The coarse density is upsampled by a factor of two in each direction (piecewise constant).
2. One pass of the fine rung's density filter smooths it, so boundary elements take intermediate values. Without this step an upsampled 0/1 design is saturated under the sharp final projection, and no small move can change it.
3. A constant density offset, found by bisection, makes the projected design volume equal the target. Keep-in and keep-out regions voxelize differently at the two resolutions, so the upsampled volume is not exactly the target.

The warm source is chosen automatically. A solve on a rung uses the design of the same specification one doubling coarser, if there is one (even a solve still running, behind which it queues). After an edit, when no coarser design of the new specification exists, it uses the latest design on the same grid. Without either, the solve is cold.

7.3 Release policies

After the warm start, the optimizer runs a continuation schedule. Table 7.2, generated from the code, lists the four schedules. Each row runs until the given iteration and sets the SIMP penalty p, the projection sharpness β, the move limit and the filter radius $r_{\min}$ in element units. All runs stop at convergence or at 300 iterations.

Tail (**Release**, the default warm start from a coarser rung). Only the final phase: $p = 4.5$, $\beta = 32$, move 0.05. It takes about 12 iterations and mostly *certifies* the upsampled design on the fine grid rather than improving it.

Long re-opening (**Optimize release**, or `optimize=True` in `solve`). The cold schedule from its second phase on: $p = 3.5$, $\beta = 4$, move 0.15, then sharpening to the final phase. The projection starts far from saturation, so boundaries move and members finer than the coarse rung can form. About 60 iterations.

Short re-opening (automatic after an edit, and for clarification previews). β steps 8, 16, 32 with larger moves, so a design on the same grid can follow a changed specification.

Cold. The full schedule from uniform density: what a solve does without a warm source, and the reference in the paper's comparisons.

Table 7.2. Continuation schedules (from `solver.py` and `escalation.py`).

Policy	Until it.	p	β	Move	$r_{\min}$	Starts from
Cold (reference)	15	1.5	1	0.20	1.50	uniform density
	40	3.5	4	0.15	1.35	
	65	4.5	16	0.08	1.25	
	120	4.5	32	0.05	1.20	
Tail	end	4.5	32	0.05	1.20	warm from the rung below (default)
Short re-opening	4	4.5	8	0.10	1.20	warm, same grid after an edit
	10	4.5	16	0.08	1.20	
	end	4.5	32	0.05	1.20	
Long re-opening	25	3.5	4	0.15	1.35	warm from the rung below, Optimize release
	50	4.5	16	0.08	1.25	
	end	4.5	32	0.05	1.20	

A tail release is not a fine-grid optimization. A sharply projected upsampled design can be saturated away from the projection threshold; continuing only the final phase then changes essentially no solid or void states. A certified tail release is an acceptance of the transferred coarse design on the fine grid, which is useful, but it is not an optimization from a uniform fine-grid start. In the paper's comparison the binary design changed essentially not at all after a tail release. Use **Optimize release** for the final design.

Release policies on five problems (paper, Table 8). At their release grids of 25.6–73.7 million elements, every final result below was certified. ΔC is the compliance relative to a cold run on the same grid; compute is the share of the cold run's release-rung compute (earlier rungs excluded) [C3.1].

Problem	Elements	Cold	Tail ΔC	Tail compute	Long ΔC	Long compute
Cantilever	65.5M	118 s	+0.28%	7%	−1.20%	69%
Bridge	65.5M	249 s	+1.39%	11%	−1.19%	51%
Torsion	65.5M	121 s	+0.86%	6%	+0.07%	59%
L-bracket	25.6M	251 s	+0.00%	3%	−3.06%	51%
Motor mount	73.7M	423 s	+1.22%	8%	+0.03%	61%

The long re-opening reached a different local optimum from the cold run (binary overlap 0.86–0.97); a lower compliance is evidence of a different trajectory, not of a global optimum. The bridge's cold reference needed a different solver configuration after two failures. A sixth problem, a duct bridge, gave no usable comparison and is excluded.

7.4 The release certificate

Every finished solve carries a certificate, created inside the solver from its own evidence. The tool contract can read it but has no way to write it. The certificate is the conjunction of six predicates:

$$A_{\text{num}} = A_{\text{conv}} \wedge A_{\text{res}} \wedge A_{\text{no spill}} \wedge A_{\text{monitor}} \wedge A_{\text{vol}} \wedge A_{\text{passive}}. \tag{7.1}$$

The final state solve is accepted on the true relative residual, recomputed in FP64:

$$r_{\text{true}} = \frac{\|f - Ku\|_2}{\|f\|_2} \leq \tau, \tag{7.2}$$

where τ is the solve's configured tolerance (normally 10^{-5} for optimization in the paper's experiments). Table 7.3, generated from `solver.py` and the result card's labels, lists the six predicates.

Table 7.3. The six certificate checks. A result is certified only when all six pass.

Check (record key)	Result-card label	Predicate	Passes when
`converged`	converged	A_{conv}	The optimizer met its stopping criterion within the iteration cap. code: `bool(res.get('converged'))`
`state_solve_accepted`	FP64 state solve accepted	A_{res}	The final state solve was accepted on the true relative residual, recomputed in FP64, at the configured tolerance τ. code: `rel is not None and rel <= self.cfg.tol`
`no_vram_spill`	no VRAM spill	$A_{\text{no spill}}$	No growth of shared GPU memory beyond the spill limit was observed: the solve stayed in dedicated GPU memory. code: `not self.spill.spilled`
`spill_monitored`	spill monitored	A_{monitor}	Spill sampling ran during the iterations. Only the release rung enables it, so lower rungs fail this check and are never certified. code: `self.cfg.spill_check_every > 0`
`volume_ok`	volume within 0.1%	A_{vol}	The largest recorded volume error is at most 10^{-3} (0.1%). code: `res.get('max_vol_error') is not None and res['max_vol_error'] <= 0.001`
`passive_exact`	keep-in/out exact	A_{passive}	Every keep-in element has physical density exactly 1 and every keep-out element exactly 0. code: `bool(pc.get('keep_in_ok', True) and pc.get('keep_out_ok', True))`

Why lower rungs are never certified. Spill monitoring samples GPU memory every 10 iterations, and only release-rung solves enable it. A sketch, preview or refine result therefore fails *spill monitored* and is shown as **not certified**, even when every other check passes (Figure 3.1). It is feedback, not a release. Only a certified result on the release rung is a released design, and the card says so whenever the latest result is on a lower rung.

What the spill check means. On a desktop GPU, the driver can move memory into shared system memory when the card fills up. A solve then still runs, but its timings no longer mean anything. The spill guard compares shared GPU memory against its value at the start of the run and stops the solve when it grows past the limit. A passing check means no spill was observed under that sampling policy.

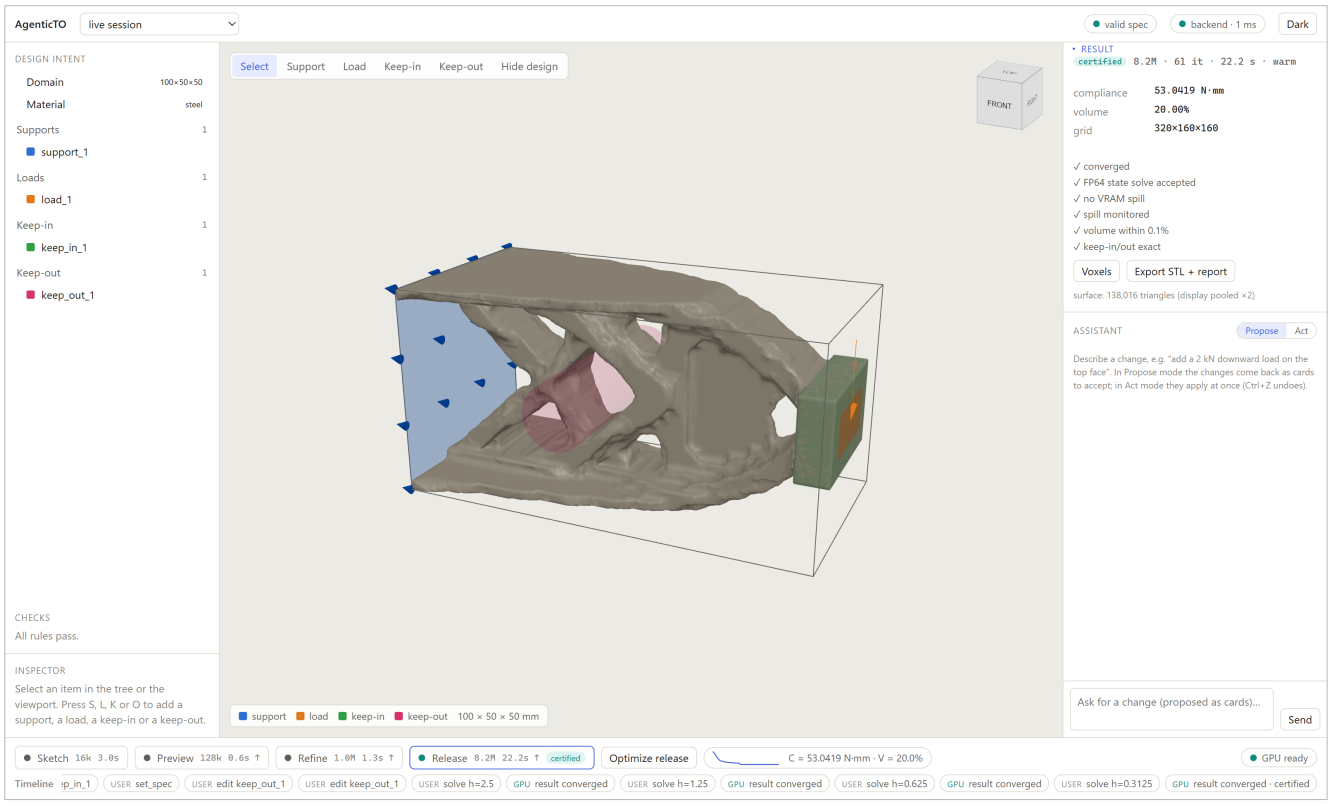


Figure 7.2. A certified release of the bracket: the marching-cubes surface of the 8.2-million-element design (on the reduced ladder), the result card on the right and every rung's time on the ladder.

> **What the certificate does not say.** The certificate belongs to the density problem that was solved, on the reported grid. It is not a proof of a global optimum. Extracting a surface does not transfer the certificate to a new mechanical analysis of that surface, and the certificate says nothing about stress, buckling, fatigue or manufacturing. In the paper's gate ablation, a problem with its load inside a keep-out region converged and passed all six numerical checks; only rule **R008** stops it [C4.4].

7.5 Surface, voxels and stale results

A certified release is shown as its $\rho = 0.5$ isosurface, extracted with marching cubes (VTK Flying Edges). For large grids the surface on screen is pooled for display, as the note under the buttons says; the exported STL is not. **Voxels** shows the raw density field (Figure 7.3).

Each result is tied to the hash of the specification it solved. Change the specification after a solve and the card marks its result **stale** (Figure 7.3): the design on screen no longer matches the specification. Undo the change and the badge clears.

7.6 Export

Export STL + report (or the tools `export_stl` and `export_report`) writes three files to the session's `exports/` folder:

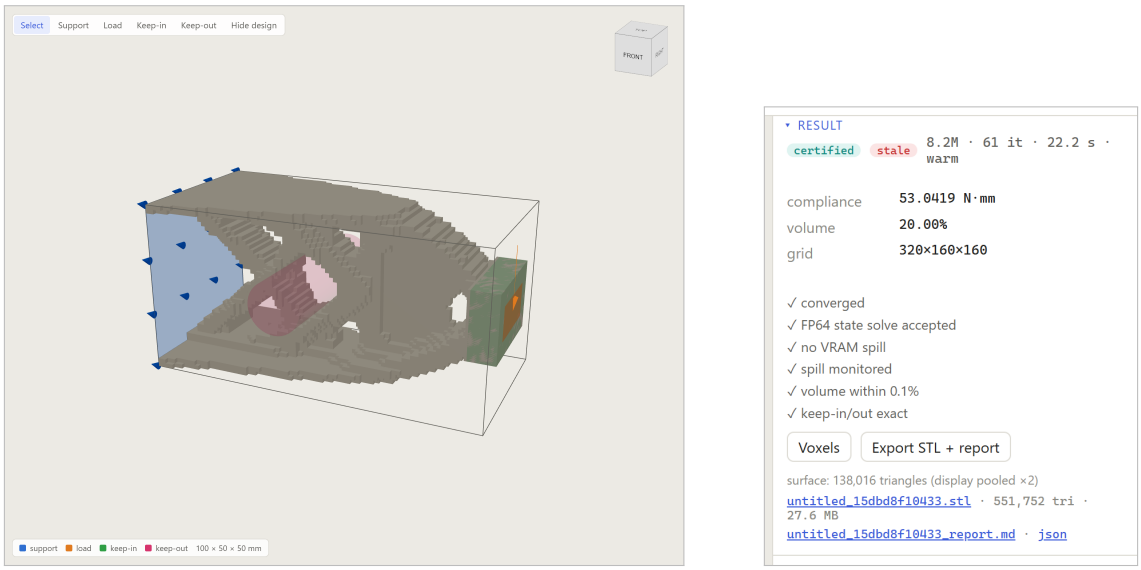


Figure 7.3. Left: the same design shown as density voxels. Right: a stale result after the volume fraction was edited.

- a closed binary STL of the full-resolution $\rho = 0.5$ surface, positions in the spec's length unit; the export also checks, for surfaces of up to 30 million triangles, that every edge is shared by exactly two triangles and records the enclosed volume;
- a Markdown report and the same report as JSON, with the specification and its hash, the rung and solver, iterations and compute time, the compliance, every certificate check, the STL record with its SHA-256, and the hash of the session log (Appendix C).

A design that is not a certified result is exported with `_draft` in its file name.

8

Working with the assistant

The assistant panel connects the workspace to a Codex session on your ChatGPT subscription. It builds and edits the specification through the same tools you use, grounds every region, and asks when a brief leaves a design-changing fact open. This chapter covers the two autonomy modes, clarification questions with consequence previews, proposal cards, and what the paper's benchmark says about when this helps.

8.1 What the assistant can do

Describe a part or a change in words, or paste a whole design brief, and press Enter. Each message starts one headless Codex session (model `gpt-6.1-sol`, reasoning effort high) that has exactly one tool server: the AgenticTO tool contract. It has no shell, no file access beyond a read-only sandbox, and no other tools, so everything it does is a logged tool call.

The prompt it receives contains its instructions, the current specification, the item you have selected (so "this" and "it" refer to your selection), the current rule findings, the latest solve result, and the last turns of the conversation. Its instructions tell it to:

- ground every region with `list_regions` and run `validate_spec` after each change, fixing what it reports;
- keep the ids that already exist;
- when the request is ambiguous in a way that changes the design (which face, how large, what force), not guess but call `ask_clarification` once with two to four concrete readings, then apply the reading you choose;
- finish with one to three plain sentences: what it changed and anything it assumed.

8.2 Propose and Act

The dial in the panel header sets the autonomy (Table 8.1).

> **An instruction, not a lock.** In Act mode the restriction to sketch and preview is part of the assistant's instructions. The `full` profile it uses does contain `solve` for every rung. Even if an agent did release a design, the certificate would still come from the solver, and you would see the solve in the timeline.

Table 8.1. The two autonomy modes.

Mode	What the assistant may do	How you see it
Propose (default)	Works on a private copy of your specification: a stand-alone session seeded with it, with the `clarify` tool profile. It can edit, ground, validate and ask; it cannot solve.	When it finishes, every difference from your specification becomes a card. Nothing changes until you accept.
Act	Attached to the live session with the `full` profile. Its edits apply at once. Its instructions allow the sketch and preview rungs to check its work and leave refine and release to you.	Edits appear immediately in the tree, the viewport and the timeline, tagged **AGENT**. **Undo this run's edits** reverts the whole run in one step.

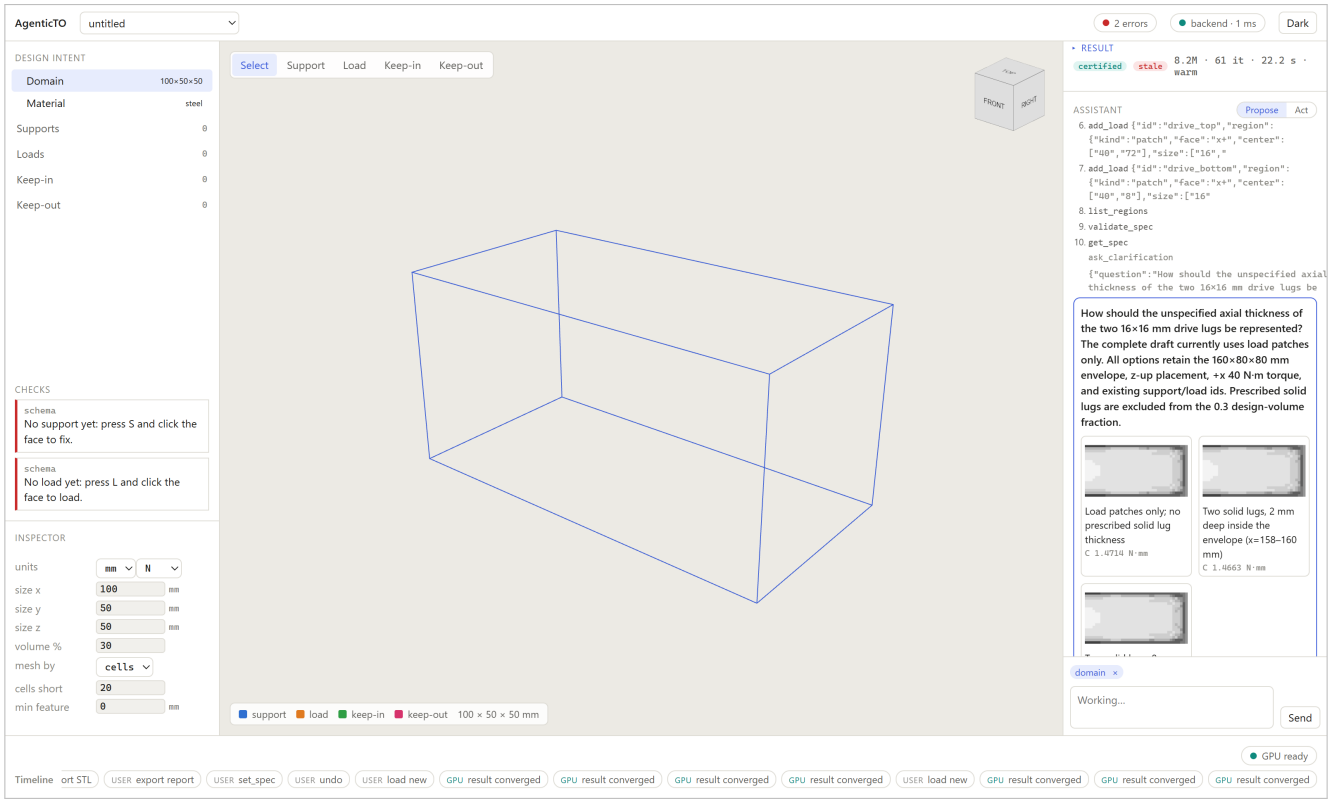


Figure 8.1. Building a part from the ambiguous brief b44, the assistant asks which axis is vertical and which way the torque turns. Each reading is shown with the design it produces. In Propose mode your own specification is untouched until you accept, so the viewport still shows the empty starting box.

8.3 Clarification questions with consequence previews

When a brief leaves something open that changes the design, the assistant asks instead of guessing. Each possible reading is a concrete option, either a complete support, load or keep-in/keep-out item, or parameter values such as a volume fraction. The workspace solves each reading on the sketch rung, all starting from the same latest design so that their compliances compare fairly, and shows it as a chip with a thumbnail of the design it leads to and its compliance (Figures 8.1 and 8.2). Pick a chip, or type your own answer under **None of these**.

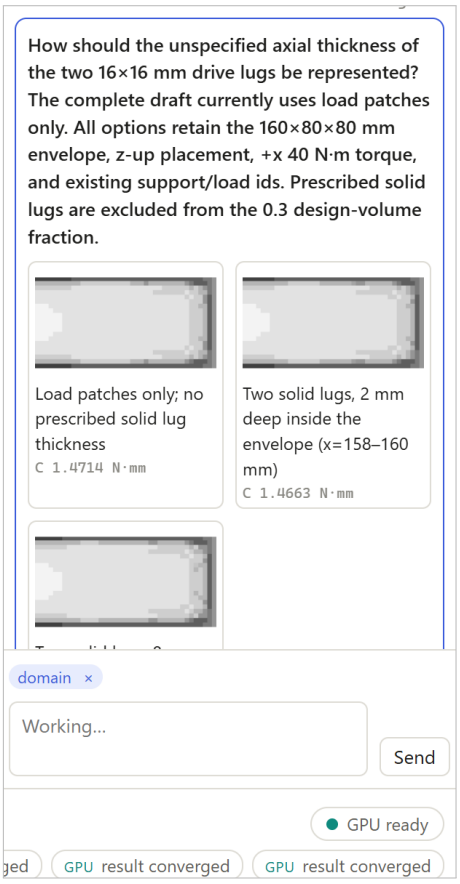


Figure 8.2. The clarification chips, each with its sketch design and compliance. Picking a chip answers the question with that reading.

The question, its options, the previews and your answer are written to the session log, so the chosen interpretation becomes part of the record, and a replay reuses the recorded answer without asking again.

> **Take your time.** A question stays open for ten minutes. The assistant is told to keep waiting for your answer. If its session has ended by the time you answer, your answer starts a follow-up run that applies it, so the answer is not lost.

8.4 Cards and the ghost overlay

In Propose mode the result is a set of cards, one per change: *add*, *change* or *remove* for an item, *set* for a field such as the volume fraction. Hover a card to see its ghost in the viewport (Figure 8.3). **Accept** or **Reject** each card, or use **Accept all** and **Reject all**. Accepted cards are applied as one undoable step, logged as yours (actor `user`), with the cards and the agent run recorded beside it.

8.5 An Act run

In Figure 8.5 the assistant in Act mode lowers the volume fraction and runs a sketch to check the design. The steps drawer lists every tool call. A failed call shows its error, which

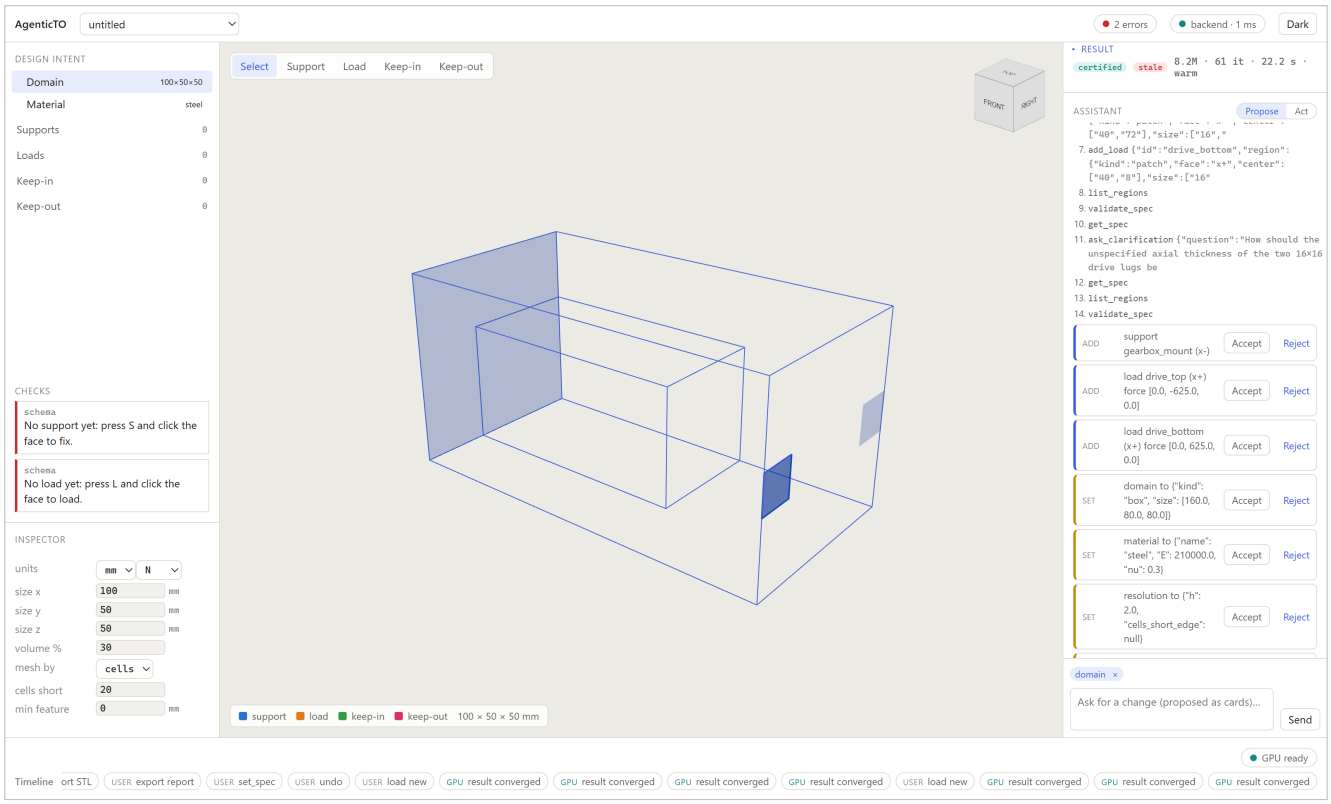


Figure 8.3. Proposed changes as cards. Hovering a card draws its ghost in the viewport.

the assistant then fixes. The footer gives the wall time and the token use. **Undo this run's edits** reverts the run as a whole.

8.6 When the assistant helps, and what it costs

The paper measured how often agents recover the intended specification from a brief, on a suite of 50 three-dimensional briefs: 40 to solve, 7 that need a clarification, and 3 that must be refused because they are ill-posed as stated.

Specification grounding on 50 briefs, primary provider, first repeat. Right behaviour means a specification equivalent to the intended or an admissible reading, or a refusal where the brief must be refused [C2.1, C2.3].

Condition	Right behaviour	Silent wrong
T: text only, JSON from the brief and schema	37 / 50	1
G: grounding tools, no questions allowed	44 / 50	6
G+C: grounding and clarification	50 / 50	0
G+C+V: G+C followed by a three-view picture check	50 / 50	0

T to G corrected 8 tasks and broke 1 ($p \approx 0.039$); G to G+C corrected 6 and broke none ($p \approx 0.031$) [C2.2]. A deterministic rule parser without a language model got 9 right.

Three lessons for use:

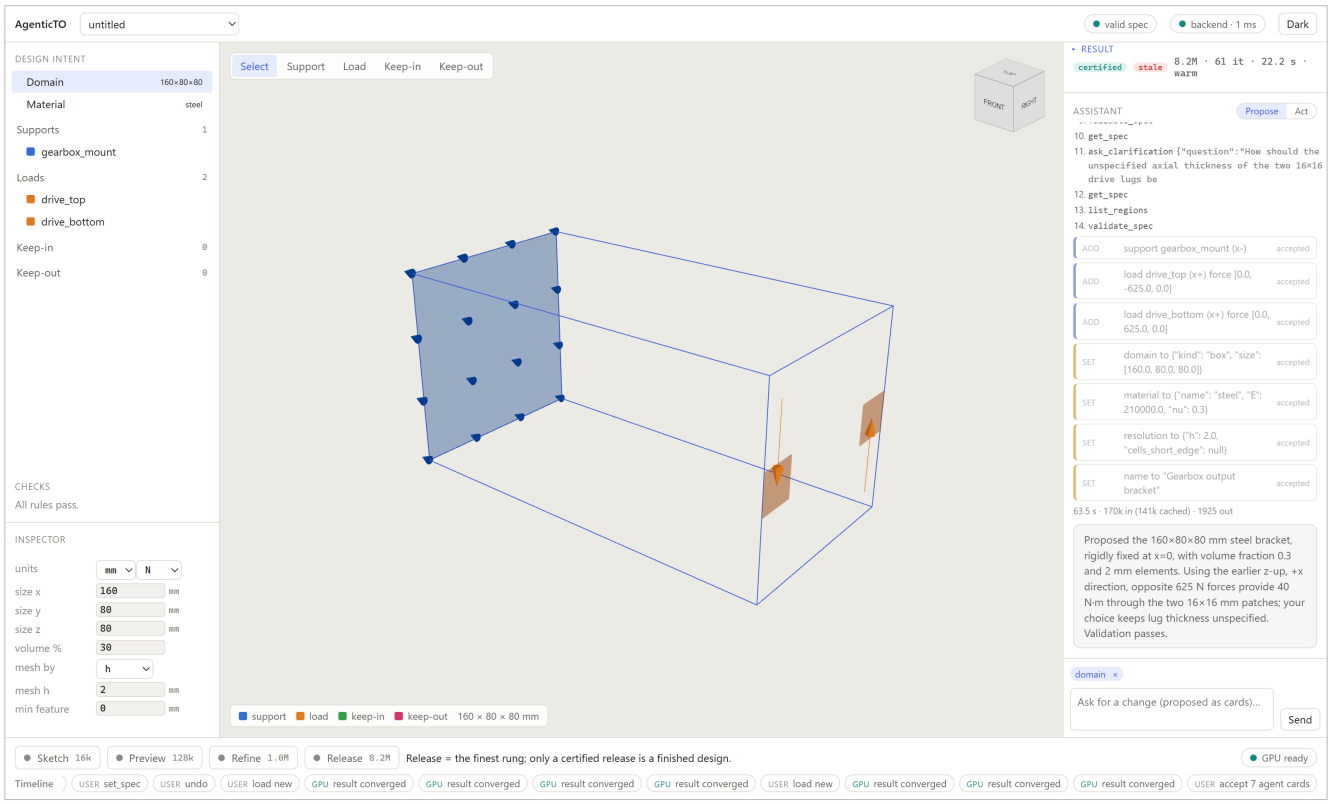


Figure 8.4. After **Accept all**: the specification is built and valid, and the ladder is ready.

1. **Grounding fixes representation errors.** Text-only output often failed to encode valid geometry (for example zero-length cylinders for washers). With the tools these errors largely disappear.
2. **Clarification fixes intent errors, at a price.** The G condition was not allowed to ask and guessed; its 6 silent errors all passed the rules. Allowed to ask, the agent fixed all six, but it asked 59 questions, 45 of them on fully stated briefs [C2.5]. Expect to confirm details that the brief already states.
3. **The benefit depends on the agent choosing to ask.** A second provider (Claude Code, `claude-opus-5-5`, effort high) with the same tools asked 7 questions instead of 59 and reached 46 of 50 under G+C, keeping 4 silent wrong specifications; two of them were unit slips. The difference in right outcomes was not significant on this sample ($p \approx 0.12$) [C2.6].

Read the cards, not the summary. An agent can explain its assumption fluently and still return a specification that differs from what you meant. The cards, the tree and the viewport show what the specification actually says. Accept a card only when you would have written that item yourself.

What the benchmark does and does not show. In the benchmark a scripted oracle answered the questions from the stored reference reading; it ignored the consequence previews. The results therefore show that asking and using the answer works under a cooperative oracle, not that the previews improve a person's choice. The reference readings were adjudicated by two authors, the second blind to condition and provider, and no participant study has been run.

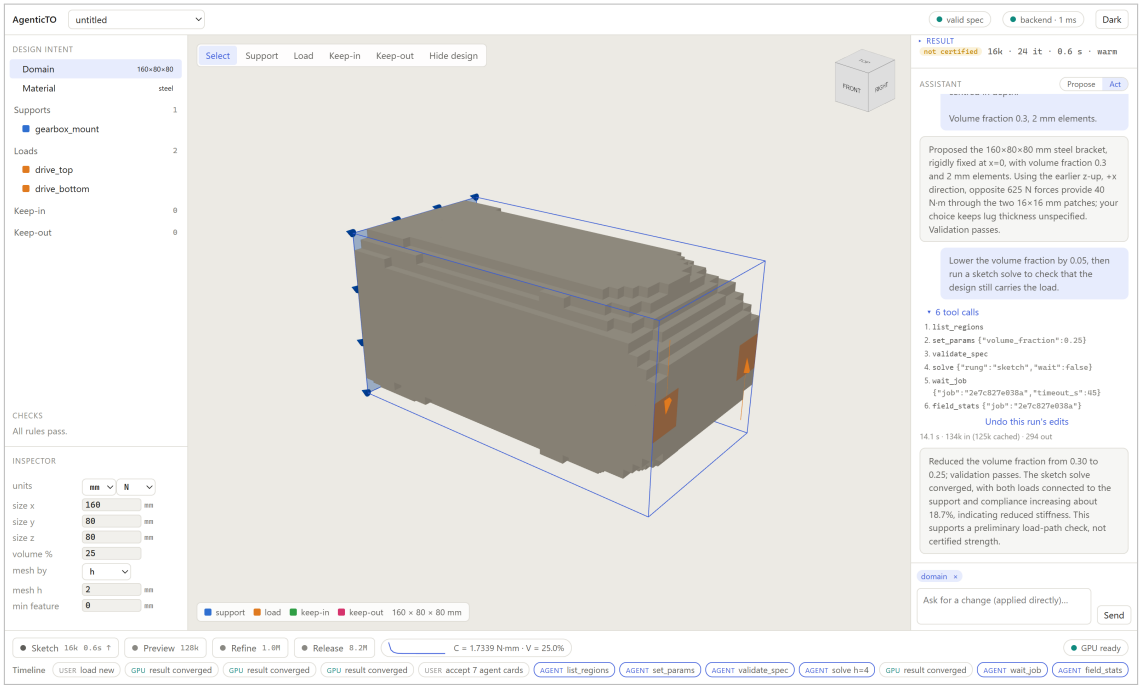


Figure 8.5. An Act run: six tool calls (list the regions, set the volume fraction, validate, solve the sketch, wait, read the field statistics), the 16,000-element sketch in the viewport, and **AGENT** records in the timeline. The run took 14 s.

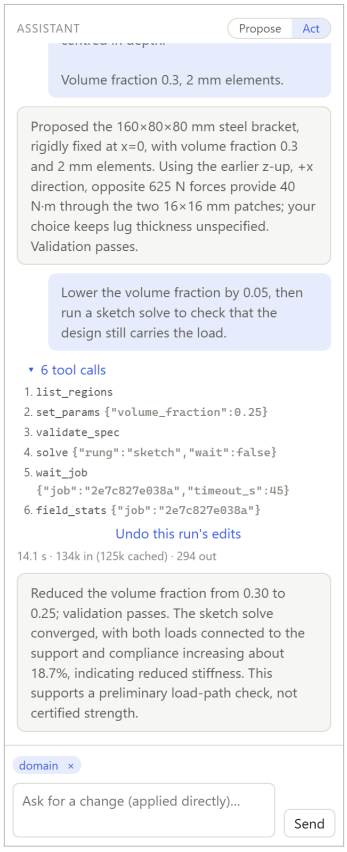


Figure 8.6. The assistant panel after an Act run: the instruction, the steps drawer with the six numbered tool calls and their arguments, the link that undoes the run, the footer with wall time and token use, and the reply.

9

Connecting external agents through MCP

The tool contract is exposed as a Model Context Protocol server, `agentto.mcp_server`. Any MCP client can drive a design session with it: attached to the live workspace, where its edits appear as you watch, or in a session of its own. This chapter covers the server's modes and profiles, configuration for Claude Code and the Codex CLI, and the conventions an agent needs to know.

9.1 The server

The server speaks MCP over standard input and output and publishes one MCP tool per contract tool, with the tool's docstring as its description and its signature as the argument schema. Every call is forwarded to a `Session` and validated there, so a malformed call is rejected and logged like any other failed call, and the agent receives the error text. Results are compact JSON text; `render` returns its picture as an image.

The server runs in one of two modes:

Attach (`--attach http://127.0.0.1:8765`). The agent drives the session the workspace has open. Its calls go to the backend's `/api/tools/`⟨*name*⟩ endpoint with actor `agent`, appear in the workspace as they happen, and can be undone like any other step. Solves run on the workspace's GPU worker.

Stand-alone (`--session DIR`). The server keeps its own session in a directory, with its own log and its own GPU worker. This is how the paper's benchmarks ran.

Command-line flags

Flag	Values	Meaning
`--session`	directory	session directory (stand-alone mode)
`--attach`	URL	backend URL of a running GUI session
`--profile`	`spec` | `clarify` | `spec_prov` | `clarify_prov` | `full` (default `spec`)	which tool set the server offers (the profile table above)
`--seed`	JSON file	stand-alone mode: JSON spec the session starts from (logged as actor system)
`--oracle`	JSON file	stand-alone mode: intended-reading JSON; a scripted oracle answers ask_clarification (X2 condition G+C)

`--answer-url`	URL	stand-alone mode: a GUI backend that puts ask_clarification to its user
`--no-rules`	JSON file	stand-alone mode: validate_spec checks the schema only, no rule findings (X2 ablation GNR)

9.2 Profiles

A profile decides which tools the server offers. The default is `spec`; give `--profile full` for an agent that should solve and export.

Profile	Tools	Contents
`spec`	16	`new_spec`, `set_spec`, `set_params`, `add_support`, `add_load`, `add_passive`, `update_item`, `remove_item`, `undo`, `redo`, `get_spec`, `list_regions`, `validate_spec`, `explain_spec`, `render`, `ladder`
`clarify`	17	everything in `spec`, plus `ask_clarification`
`full`	27	everything in `clarify`, plus `load_template`, `set_source`, `list_sources`, `solve`, `status`, `wait_job`, `cancel`, `field_stats`, `export_stl`, `export_report`

In the paper, `spec` was the grounding condition G, `clarify` the condition G+C and the Propose assistant, and `full` the complete sessions from brief to release, Act mode, and attaching to the workspace. The follow-up conditions used two further profiles. `spec_prov` adds `set_source` and `list_sources` to `spec` (condition G+P): the agent tags each specification item as stated, derived, assumed or answered. `clarify_prov` does the same for `clarify` and limits questions to items tagged assumed (condition G+C+P). Under both, `validate_spec` adds an advisory note with code R014 that lists the items tagged assumed and the untagged items; it never blocks a solve and is not one of the thirteen rules. The `--no-rules` flag gave the rule-feedback ablation GNR.

9.3 Attach Claude Code to the workspace

With the backend running, save this as `mcp.json`, replacing the two paths with your Python interpreter of the `agentto` environment and the repository's `src` directory:

```
{"mcpServers": {"agentto": {
  "command": "/path/to/envs/agentto/bin/python",
  "args": ["-B", "-m", "agentto.mcp_server", "--profile", "full",
           "--attach", "http://127.0.0.1:8765"],
  "env": {"PYTHONPATH": "/path/to/AgenticTO/src"}}}}
```

```
claude --mcp-config mcp.json
```

Allow the `mcp__agentto` tools when Claude Code asks, or start it with `--allowedTools mcp__agentto`. Set `AGENTTO_VOXFEM_SRC` in the environment block too if the server runs stand-alone and must solve.

In Figure 9.1, Claude Code (`claude-opus-5-5` on a claude.ai subscription) was asked to add a 300 N load on the top face of the part from Chapter 8 and to validate the specification. It

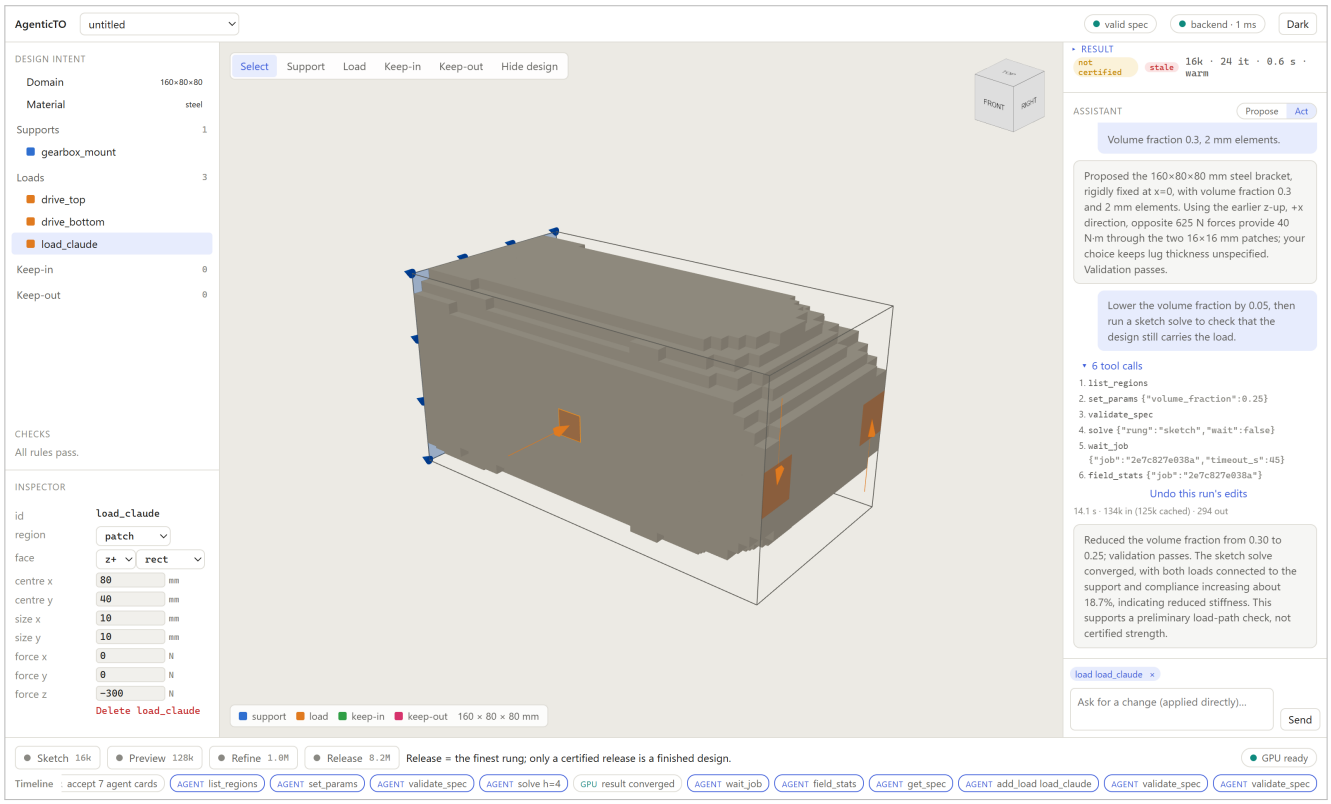


Figure 9.1. Claude Code, attached over MCP, added `load_claude` to the live design.

took 13 s. Its calls appear in the timeline as **AGENT** records, and the new load appears in the tree and the viewport.



Figure 9.2. The timeline after the Claude Code session: its tool calls are tagged **AGENT**.

9.4 Attach the Codex CLI

Add this to `~/.codex/config.toml`:

```
[mcp_servers.agentto]
command = "/path/to/envs/agentto/bin/python"
args = ["-B", "-m", "agentto.mcp_server", "--profile", "full",
        "--attach", "http://127.0.0.1:8765"]
env = { PYTHONPATH = "/path/to/AgenticTO/src" }
startup_timeout_sec = 60
tool_timeout_sec = 900
default_tools_approval_mode = "approve"
```

Approval in non-interactive runs. Without `default_tools_approval_mode = "approve"`, non-interactive Codex runs (`codex exec`) reject every MCP call, because they cannot ask you. The AgenticTO tools only touch the session directory. A `tool_timeout_sec` shorter than your longest solve cuts the call off; the solve itself keeps running, and the agent can collect it with `wait_job`.

9.5 Stand-alone sessions

Without the workspace, the server keeps its own session:

```
python -B -m agentto.mcp_server --session runs/my_session --profile full
python -B -m agentto.mcp_server --session runs/b01 --profile clarify --seed spec.json
python -B -m agentto.mcp_server --session runs/b01 --profile clarify \
    --answer-url http://127.0.0.1:8765
```

`--seed` starts the session from an existing specification (logged as actor `system`). `--answer-url` sends the agent's clarification questions to the user of a running workspace; this is how the Propose assistant asks you its questions. `--oracle` makes a scripted oracle answer them from a reference reading (the benchmark's condition G+C); without an answerer, a question returns at once with the advice to choose the most reasonable reading and state it as an assumption.

9.6 Conventions an agent needs

The server's instructions and the tool descriptions carry most of these; they are listed here so that you can write your own prompts.

Coordinates. Absolute, origin at the domain's minimum corner. Patch centres and sizes are in the face's two in-plane axes in increasing order. `new_spec` and `list_regions` return the in-plane axes of every face.

Grounding. After adding an item, `list_regions` reports how many nodes or elements it selects; an item that selects 0 does nothing. `validate_spec` returns the rule findings. `render` returns a picture to check placement and directions.

Long solves. `solve` blocks until the job finishes or `timeout_s` (default 900 s) passes. On a timeout it returns the job id with a hint; the agent should call `wait_job`, not cancel and resubmit, which would start the run again from zero. `wait=False` returns the job id at once; `status` lists the jobs.

Rungs. Ask for a rung by name. The release rung is often finer than the specification's `h`, which still satisfies a brief that asks for elements of at most `h`. Asking for a rung and an `h` that disagree is an error.

Certificates. The agent can read the certificate in every solve result, but no tool can write one. Only a certified release-rung result is a released design; anything else is exported as a draft.

Clarification. One question at a time, with two to four options, each a complete item or parameter values. The call returns once the user has answered.

The complete tool reference, with every argument, its type and its default, is Appendix B. The summary below shows which tools change the specification, which profiles offer them, and how replay treats them.

Tool	Changes spec	`spec`	`clarify`	`full`	Replay
Build the spec					

`new_spec`	yes	✓	✓	✓	re-run, compared
`set_spec`	yes	✓	✓	✓	re-run, compared
`load_template`	yes	–	–	✓	re-run, compared
`set_params`	yes	✓	✓	✓	re-run, compared
`add_support`	yes	✓	✓	✓	re-run, compared
`add_load`	yes	✓	✓	✓	re-run, compared
`add_passive`	yes	✓	✓	✓	re-run, compared
`update_item`	yes	✓	✓	✓	re-run, compared
`remove_item`	yes	✓	✓	✓	re-run, compared
`undo`	yes	✓	✓	✓	re-run, compared
`redo`	yes	✓	✓	✓	re-run, compared
Inspect and check					
`get_spec`	–	✓	✓	✓	re-run, compared
`list_regions`	–	✓	✓	✓	re-run, compared
`validate_spec`	–	✓	✓	✓	re-run, compared
`explain_spec`	–	✓	✓	✓	re-run, compared
`render`	–	✓	✓	✓	re-run, compared
`ladder`	–	✓	✓	✓	re-run, compared
Ask the user					
`ask_clarification`	–	–	✓	✓	re-run, compared
Record value sources					
`set_source`	–	–	–	✓	re-run, compared
`list_sources`	–	–	–	✓	re-run, compared
Solve					
`solve`	–	–	–	✓	re-solved, compared
`status`	–	–	–	✓	skipped
`wait_job`	–	–	–	✓	skipped
`cancel`	–	–	–	✓	skipped
`field_stats`	–	–	–	✓	re-run, compared
Export					
`export_stl`	–	–	–	✓	re-run, compared
`export_report`	–	–	–	✓	re-run, compared

9.7 The HTTP interface

The backend's own endpoints are listed below, generated from `src/agentto/app/server.py`. Attach mode uses only `POST /api/tools/⟨name⟩` with the body `{"args": {...}, "actor": "agent"}`; the workspace uses the WebSocket. The backend listens on `127.0.0.1` only.

Method	Path	Purpose (from the handler's docstring or name)
GET	`/api/health`	Version and schema version.
GET	`/api/schema`	The live JSON Schema of the spec.
GET	`/api/library`	Names of the template problems.
GET	`/api/library/{name}`	A template problem as a spec (query parameter k: cells on the short edge).

GET	`/api/tasks`	Draft X2 tasks (briefs/intended/*.json), for development: their intended specs load like library items.
GET	`/api/tasks/{slug}`	A suite task: its intended spec, expected response and brief text.
POST	`/api/validate`	Run the schema and the rules on a posted spec.
POST	`/api/ladder`	The rungs for a posted spec.
POST	`/api/tools/{name}`	Tool contract over HTTP (used by the MCP server in --attach mode).
GET	`/api/session`	The live spec, the tail of the session log and the chat.
POST	`/api/clarify`	ask_clarification from a stand-alone (propose-mode) agent session: answered by this app's user.
GET	`/api/mesh/{job}`	Preview surface of a finished design (pooled to at most 1.5M triangles): uint32 n_vertices, uint32 n_triangles, float32 h_eff, then float32 positions (spec units) and uint32 indices.
GET	`/api/files/{name}`	Download an exported file (STL, report) of the live session.
WS	`/ws/session`	JSON messages {type, ...}: ping, validate, tool {name, args, meta, ref}, worker_start.

10

Logs, replay and provenance

Every session writes a log of everything that happened in it, by whom, and to which specification. This chapter describes the session folder, the four record types, the specification hash, the replay that re-runs a log without any language model, and the tool contract from Python.

10.1 The session folder

The backend writes each workspace session to `sessions/`⟨*yyyymmdd-hhmmss*⟩`/` in the repository root. A stand-alone MCP session or a script writes to the directory you give it. The layout is the same:

Table 10.1. The session folder.

Path	Contents
`session.jsonl`	The log: one JSON record per line (Section 10.2).
`fields/`	The final density of every finished solve, `rho_`⟨*job*⟩`.npy`, as float16 (2 bytes per element). `field_stats`, the surface view and the export read them.
`exports/`	STL files and design reports.
`renders/`	The PNG pictures made by `render`, named by call number and view.
`agent/`⟨*run*⟩`/`	One folder per assistant run: the run's Codex event stream (`work/events.jsonl`), its standard error, its final message and, in Propose mode, the seed specification and the private session it worked in.

10.2 Records

The log has four record types. The examples below are real records from the paper's agent session on brief b06 (`results/X3/b06_motor_mount/session/session.jsonl`), reformatted and shortened.

`session` opens the log: time, software version, schema version, a session id and the tool profile.

```
{"type": "session", "t": "2026-10-30T12:19:37.880+00:00", "agentto": "0.0.1",
 "schema": "agentto.spec/v0", "id": "f5f023ceb534", "profile": "full"}
```

`call` records one tool call: its sequence number, time, actor, tool, arguments, whether it succeeded, its result (or error), the hash of the specification after the call, and its duration. Binary payloads such as PNG images are dropped from the log; their hashes are kept.


```
{"type": "call", "seq": 2, "t": "2026-10-30T12:20:00.035+00:00", "actor": "agent",
 "tool": "set_params",
 "args": {"material_name": "Aluminium 6061", "E": 69000, "nu": 0.33, "h": 1,
          "volume_fraction": 0.2},
 "ok": true, "spec_hash": "be59dc4a6544fc93", "dt_s": 0.0001,
 "result": {"changed": ["E", "nu", "material_name", "volume_fraction", "h"]}}
```


`result` records a solve that finished after its call returned: the job, its grid and solver, whether it was warm-started and from which call, iterations, compliance, volume, compute and wall time, the certificate verdict and its six checks, and the hash of the specification it solved. This one is the cold sketch; note that `spill_monitored` is false, so it is not certified.


```
{"type": "result", "seq": 25, "job": "39cd686c8342",
 "result": {"status": "converged", "h": 2.0, "grid": [80, 30, 60],
   "n_elem": 144000, "solver": "mg64", "warm_used": false, "iterations": 80,
   "compliance_phys": 3.9057077848391675, "volume": 0.199999999999788,
   "compute_s": 6.29, "wall_s": 6.67, "certified": false,
   "checks": {"converged": true, "state_solve_accepted": true,
              "no_vram_spill": true, "spill_monitored": false,
              "volume_ok": true, "passive_exact": true},
   "release_rung": false, "spec_hash": "c9bd97c53d933b5b"}}
```


`answer` records the answer to a clarification question (who answered, which option or none, and any comment) and, when previews were computed, their summaries.


```
{"type": "answer", "seq": 3, "t": "2026-10-30T12:20:00.065+00:00",
 "answer": {"choice": null, "label": null, "by": "oracle",
            "comment": "None of these is what the brief means."}}
```


The actor is `user` for workspace edits, `agent` for the assistant and attached MCP clients, `system` for a seed, and `script` for Python calls unless you pass another name. A workspace call also carries a `meta` field with its origin and a human-readable label (for example “accept 3 agent cards”); replay ignores it.

10.3 The specification hash

`spec_hash` is the first 16 hexadecimal digits of the SHA-256 of the specification in canonical JSON (sorted keys, no spaces). A schema-valid specification is first normalized through the model, so default values count and two documents that differ only in omitted defaults hash the same. Every call records the hash after it; every solve records the hash it solved; every report records the hash of its specification. This is how the workspace knows that a result is stale, and how anyone can check later that a released design belongs to a given specification.

The report also records the SHA-256 of `session.jsonl` at the moment of export, which ties the release to the full history that produced it.

10.4 Replay without a language model

```
import sys; sys.path.insert(0, "src")
from agentto.tools import replay
print(replay("sessions/20261031-101500/session.jsonl", root="runs/replayed"))
```

Replay starts a fresh session and re-runs every recorded call in order, with the recorded arguments and actors. It checks:

- every call's success flag and the specification hash after it;
- every `render` picture's hash and every exported STL's hash;
- every clarification, by reusing the recorded answer instead of asking;
- every finished solve, which must reproduce its status, iteration count, certificate verdict, warm start and compliance, bit for bit.

Calls whose outcome depends on timing (`status`, `wait_job`, `cancel`) are skipped, as are solves that were cancelled or superseded. The result lists the number of calls, the solves replayed, the calls skipped, every mismatch, and the final specification hash:

```
{"ok": true, "calls": 36, "solves_replayed": 5, "skipped": 3, "mismatches": [],
 "final_spec_hash": "c9bd97c53d933b5b"}
```

(the replay record of the b06 session above).

Why the solves reproduce bit for bit. The upstream solver assembles its coarsest multigrid matrix with atomic scatter-add operations, whose reduction order varies from run to run, and at the sharpest projection a difference of 10^{-12} can flip single elements of the final design. Sessions use a deterministic coarsest assembly with a fixed reduction order instead. Each solve also records the coarse-level policy it used, and replay re-runs it with that policy.

From the paper. All 15 complete agent sessions of the paper's study X3, including the one that ended in a refusal, replayed with no mismatch [C1.4]. Replay checks that the recorded decisions reproduce the recorded results. It does not show that a fresh language-model run would make the same decisions, and it does not check that the decisions matched the engineer's intent.

10.5 The tool contract from Python

Scripts use the same contract. Each `call` is validated, applied and logged exactly as a workspace edit or an agent call would be, and returns `{"ok", "seq", "spec_hash", "result"}` or an `"error"` in place of the result.

```
import sys; sys.path.insert(0, "src")
from agentto.tools import Session

if __name__ == "__main__":                      # the GPU worker is a spawned process
    s = Session("runs/demo", profile="full")    # writes runs/demo/session.jsonl
    s.call("new_spec", {"name": "bracket", "size": [100, 50, 50]})
    s.call("set_params", {"volume_fraction": 0.3, "cells_short_edge": 20})
    s.call("add_support", {"id": "wall", "region": {"kind": "face", "face": "x-"}})
    s.call("add_load", {"id": "tip", "force": [0, -2000, 0],
                        "region": {"kind": "patch", "face": "x+",
                                   "center": [25, 25], "size": [10, 20]}})
    print(s.call("validate_spec"))
    print(s.call("ladder"))
    print(s.call("solve", {"rung": "sketch"}))  # blocks until the sketch is done
    s.runner.close()                             # stop the GPU worker
```

A solve needs a complete specification. `new_spec` creates a draft without a volume fraction or a resolution, and the schema requires both. Until `set_params` supplies them, `validate_spec` returns schema errors and `solve` is refused with “spec is not complete/valid”.

A `Session` without a directory (`Session(None)`) keeps no log and no density fields, which is enough for building and validating specifications in a test. A script that solves must protect its entry point, as above, because the GPU worker is started as a spawned process that re-imports the script.

11

Reproducing the paper

Every number in the paper comes from a result file, and every table and figure is regenerated from those files by a script. This chapter maps each table and figure to its generator, its records and the experiment that produced them, marks which steps need the GPU or a language model, and explains the claims ledger.

11.1 Three levels

Reproduction works at three levels of cost. Each later level includes the earlier ones.

Rebuild none. Regenerate the claims ledger, every table and every plotted figure from the saved result files. Needs Python with `matplotlib`; no solver, no GPU, no language model. Takes seconds.

Replay GPU. Re-run a saved session log without a language model (Section 10.4). Needs the solver and a free GPU; the solves must reproduce bit for bit under the recorded configuration.

Re-run GPU, heavy LLM. Repeat an experiment from scratch. The numerical studies need long, exclusive use of a 24 GB GPU; the agent studies need subscription CLIs logged in as in Section 2.6. A fresh agent run makes fresh decisions, so its counts can differ from the paper's.

The badges in the tables mean: none plain Python; solver package imports the `voxfem` package but uses no GPU; GPU minutes of GPU time; GPU, heavy grids of tens of millions of elements, from minutes to hours per run; LLM Codex or Claude Code sessions on a subscription.

> **One GPU job at a time.** Every GPU script assumes it is alone on the card. A browser with hardware acceleration, a game or another CUDA job can push a large solve into shared memory; the spill guard then stops it, and timings taken beside such a job are not comparable with the paper's.

11.2 Rebuilding the tables, figures and ledger

```
python -B scripts/make_ledger.py      # results/claims_ledger.md
python -B scripts/make_tables.py      # paper/tables/<name>.tex and .md
python -B scripts/make_figures.py     # results/figures/<name>.pdf and .png
```

`make_tables.py` writes booktabs tables to `paper/tables/`; every number comes from a result file named in the table's source line, and only the rule descriptions and condition labels are typed by hand. `make_figures.py` uses the project's figure style (Arial, Okabe-Ito colours with fixed meanings, text of at least 6.5 pt at print size). The manuscript package freezes copies of these assets and builds with `latexmk -pdf`; the repository keeps its figure scripts in `paper/V5/scripts/`, with their frozen inputs.

11.3 Main paper

Paper item	Rebuilt by	Records	Produced by
Table 1, knowledge model	in the manuscript source	–	–
Table 2, rule base	`make_tables.py` (`t2_rules`)	`src/agentto/rules.py`	– none
Table 3, evaluation studies	in the manuscript source	–	–
Table 4, task suite	`make_tables.py` (`t1_suite`)	`briefs/`, `briefs/intended/`	`make_briefs.py` none
Table 5, grounding on 50 briefs, with the follow-up conditions	`make_tables.py` (`t4_x2_conditions`)	`results/X2/⟨cond⟩/*_r1.json`	`x2_run.py` LLM; `x2_rules_baseline.py` solver package; `x2_rescore.py` solver package
Table 6, two providers	`make_tables.py` (`t7_providers`)	`results/X2/claude_high/`	`x2_run.py --provider claude --effort high` LLM
Table 7, gates	`make_tables.py` (`t12_gates`)	`results/C4/census.json`, `rules_off.json`	`c4_gate_ablation.py census` none; `rules_off` GPU
Table 8, release policies	`make_tables.py` (`t3_x1_escalation`)	`results/X1/escalation.json`	`x1_escalation.py`, `x1_fix_cold.py` GPU, heavy
Figure 1, workflow	`paper/V5/scripts/make_fig1_workflow.py`	frozen `paper/V5/figures/fig1_data/`, from the b06 session in `results/X3/`	`x3_run.py` LLM GPU
Figure 2, GUI	`paper/V5/scripts/make_fig2_gui.py`	`paper/V5/figures/gui_overview.png`	`gui_screenshots.py` GPU
Figure 3, outcomes	`make_figures.py`	`results/X2/`	as Table 5
Figure 4, released designs	`paper/V5/scripts/make_fig_designs.py`	STLs of the sessions in `results/X3/`	`x3_run.py` LLM GPU; see the note below
Figure 5, session time	`make_figures.py`	`results/X3/`, `results/X3_aware/`	`x3_run.py` LLM GPU; `x3_retime_aware.py` GPU
Figure 6, release policies	`make_figures.py`	`results/X1/escalation.json`	as Table 8
Figure 7, resolution path	`make_figures.py`	`results/X5/resolution.json`	`x5_resolution.py`, `x5_chain.py` GPU, heavy

11.4 Supplementary material

Supplement item	Rebuilt by	Records	Produced by
Table S1, studies and records	in the supplement source	–	–
Table S2, paired tests	`make_tables.py` (`t5`)	`results/X2/`	as Table 5
Table S3, repeats	`make_tables.py` (`t6`)	`results/X2/⟨cond⟩/*_r2, *_r3`	`x2_run.py` LLM
Table S4, adjudication	in the supplement source	`paper/adjudication/`	`apply_adjudication.py`, `x2_rescore.py` solver package
Table S5, two adjudications	in the supplement source	`paper/adjudication/round2/`	`compare_adjudications.py` none
Table S6, schedules	in the supplement source	`src/agentto/escalation.py`, `solver.py`	– (also Table 7.2 of this book)
Table S7, part compliance	`make_tables.py` (`t9`)	`results/X5/resolution.json`	as Figure 7
Table S8, sessions	`make_tables.py` (`t8`)	`results/X3/`, `results/X3_aware/`	as Figure 5
Table S9, latency	`make_tables.py` (`t11`)	`results/X6/latency/`	`x6a_latency.py`, with the backend running GPU
Table S10, interaction model	`make_tables.py` (`t10`)	`results/X6/klm.json`	`x6b_klm.py` solver package
Table S11, rule findings	in the supplement source	first-repeat tool logs in `results/X2/`	`x2_run.py` LLM
Table S12, injected faults	in the supplement source	`results/C4/fault_matrix.json`	`c4c_fault_matrix.py` solver package
Figures S1–S3, 3D renders	`make_renders.py` (`pyvista`)	density fields and STLs of X1, X5, X3	the studies above; see the note below
Figure S4, padding	`make_figures.py`	`results/solver/`	`mg_padding_study.py`, `b19_release_aware.py` GPU
Figures S5, S6	`make_figures.py`	`results/X6/`	as Tables S9, S10

Density fields and meshes are not in the repository. The release package (`scripts/make_release.py`) leaves out density fields (`*.npy`), meshes (`*.stl`, `*.vtp`) and the render pictures inside session folders: together they are several gigabytes, and they regenerate from the session logs by replay. To redraw Figure 4 of the paper and Figures S1–S3, first replay the sessions that produced the fields.

11.5 The claims ledger

`results/claims_ledger.md` lists every number the paper may state, read from the result files by `scripts/make_ledger.py`, with its source file and a status: *supported* (the evidence says what the claim says), *partial*, *restate* (the planned wording went beyond the evidence; the row gives the wording to use), *pending* or *missing*. Never edit the ledger by hand; rerun the script. The claim identifiers in grey brackets throughout this book refer to its rows.

Group	Topic	Studies and records
C1	Verification and replay	X0 (`results/X0/`): identity with the inherited solver, independent cross-check with scikit-fem, scripted replay; X3 replays; the rule-parser baseline
C2	Specification grounding	X2 (`results/X2/`): conditions, paired tests, silent errors, repeats, questions, the second provider
C3	Scale and release policies	X1 (`results/X1/`), X5 (`results/X5/`), padding study (`results/solver/`)
C4	Gates	card replay (`results/X6/cards.json`), gate census and rule removal (`results/C4/`)
C5	Complete sessions	X3 (`results/X3/`) and its padding-aware re-timing (`results/X3_aware/`)
C6	Interface	interaction model, cards and latency (`results/X6/`)

Two rows carry the status *restate*, and the paper uses their wording: C3.2 (releases are $2.6\text{–}7.4 \times 10^7$ elements, not 1.1×10^8) and C6.5 (cold sketches miss the 1–3 s target).

11.6 Re-running the studies

The experiment scripts write their records into `results/`, beside the paper's, and most keep their session folders. Before re-running one, read its module docstring (Appendix E lists them all) and move the paper's records aside if you want to compare.

X0, verification GPU. `x0_identity_gpu.py`, `x0_export_crosscheck.py` then `x0_crosscheck_skfem.py` (needs `scikit-fem`), `x0_replay.py`, `x0_coarse_bc.py`.

X1, release policies GPU, heavy. `x1_escalation.py` runs preview, refine and release rungs with every arm; the release rungs are 26–74 million elements, and a single cold release-rung run took 118–423 s of compute on the paper's workstation.

X2, grounding LLM. `x2_run.py` with `--provider codex` or `claude`; then `x2_rescore.py` against the adjudicated readings, and `x2_score.py` GPU for the consequence-weighted error.

X3, complete sessions LLM GPU, heavy. `x3_run.py` ⟨*task*⟩ `[--provider claude]`: an agent with its own GPU worker takes a brief to a certified release, then the log is replayed.

X5, resolution GPU, heavy. `x5_resolution.py` and `x5_chain.py`; the finest grid is about 33 million elements.

X6, interface `x6a_latency.py` GPU with the backend running; `x6b_klm.py` solver package; `x6c_cards.py` solver package.

Gates `c4_gate_ablation.py census` none; `c4_gate_ablation.py rules_off` GPU.

GUI test `gui_test.py` GPU, against a backend started with `--ladder-max 1.2e7`; its `assistant` group runs Codex sessions LLM.

What a re-run can and cannot match. Numerical re-runs on the same hardware and solver configuration should match the recorded iterations and compliances; timings vary from run to run. Agent re-runs depend on the model, its version and the CLI; the paper's records name Codex CLI 0.165.0 with `gpt-6.1-sol` and Claude Code 2.1.312 with `claude-opus-5-5`, both at effort high, run on 30–31 October 2026. Reference readings and the comparator are in `briefs/intended/` and `src/agentto/bench.py`.

12

Limits and troubleshooting

What this version of AgenticTO does not do, what the paper's evidence does not establish, and what to do when something goes wrong.

12.1 Scope of the software

- **One physics model.** Isotropic, small-displacement, linear-elastic compliance minimization under one volume constraint, with one load case: all loads act together. No stress limits, buckling, fatigue, contact, nonlinear materials, multiple load cases or manufacturing constraints.
- **Box domains on a regular voxel grid.** No import of existing geometry (STL or STEP) as a design domain; complex envelopes are described with keep-out regions.
- **Element-unit filters.** The density filter radius is given in elements, so the smallest member shrinks in physical size as the grid is refined. `min_feature` only feeds a warning (**R009**); it does not impose a physical length scale.
- **GPU memory bounds the release.** The default ladder stops below 8×10^7 elements. In the paper the certified releases were 12.3–73.7 million elements in the session study and 25.6–73.7 million in the release-policy study; a guarded 1.13×10^8-element attempt did not fit beside a normal desktop's GPU memory use [C3.2].
- **Interaction set.** The workspace has no brush editing, design variants, comparison views, parameter sweeps or automatic sketch on every edit. It has been tested in Chrome on Windows with one user per session.
- **Agents.** The built-in assistant uses Codex; other agents connect through MCP. How much clarification helps depends on whether the model asks (Section 8.6).
- **Platform.** Tested on Windows 11 with an NVIDIA RTX 4090. The spill guard reads Windows GPU-memory counters.

12.2 What the evidence does not establish

The paper states these limits; they apply to every use of the numbers in this book.

- The 50 reference readings and all 41 contested responses were adjudicated by two authors, not by independent engineers; they agreed on 49 of 50 readings and 33 of 41 contested responses. Accuracy counts and paired p values stay provisional until an adjudication outside the author team exists.

- The suite is small and was developed alongside the system. Only two providers, one model each, were tested.
- A certificate is a set of numerical acceptance checks on one computation. It is not a structural safety assessment, and the exported surface needs further analysis before anything is built.
- Warm releases reach different local optima from cold runs. Neither warm policy guarantees the cold optimum or new members; a duct-bridge problem gave no usable comparison at all.
- Timings are single-workstation measurements on a shared desktop GPU. Backend latency excludes browser rendering, and the interaction model excludes waiting, errors and learning. No participant study has been run.
- Replay shows that recorded decisions reproduce recorded results. It does not show that a fresh agent run would decide the same way.

12.3 Troubleshooting

Symptom	Cause and fix
The **GPU** pill stays *starting* or turns *dead*.	The worker could not load CuPy or the solver. Check `AGENTTO_VOXFEM_SRC` and the CUDA 12 installation, then read the backend console.
The ladder is missing or greyed out.	The specification has an error, or is still incomplete. Fix what **Checks** lists in red.
A release stops with a spill message.	Another program holds GPU memory and the solve would spill into shared memory. Close GPU-heavy programs (browsers with hardware acceleration, games, other CUDA jobs), or release a coarser rung with `--ladder-max`.
A release is **not certified**.	Read the ✗ on the card. *converged* failing means the iteration cap was reached: try **Optimize release** or check the specification. *spill monitored* failing means the result is not on the release rung. A volume or keep-in/out failure points to a problem worth reporting.
The first sketch of a new specification takes several seconds.	Expected: the first solve builds the multigrid hierarchy and runs the cold schedule (2.5–6.7 s in the latency study). Later sketches after an edit are warm.
A rung takes far longer than its neighbours.	The grid may be padded for multigrid (rule **R012** may say so) or the problem converges slowly; preview and refine can reach their iteration cap. **Stop** cancels the solve.
The release grid is finer than the specification's h.	By design: the release rung is the finest that fits on the GPU. `ladder` explains this in a note. The certificate's `regularization` record gives both element edges and the filter length at each, so the shorter physical filter of the release is on record.
Rule **R013** on a large rung.	The memory estimate exceeds the GPU. Use a coarser rung, or cap the ladder.
The assistant refuses to start: `PolicyError`.	An API-key variable is set, or the CLI is not on a subscription login. Unset the variable or log in again (Section 2.6).
The assistant answers but changes nothing.	In Propose mode changes come back as cards: accept them. In Act mode, open the steps drawer and look for failed calls.
The assistant asks about details the brief already states.	Known behaviour (Section 8.6). Answer, or choose the reading that matches the brief.

An external agent's calls are all rejected.	For Codex set `default_tools_approval_mode = "approve"`; for Claude Code allow the `mcp__agentto` tools.
An agent cancels and resubmits a long solve.	Its tool call timed out. Tell it to call `wait_job`; raise the client's tool timeout.
`solve` from a script fails with "spec is not complete/valid".	Set the volume fraction and the resolution with `set_params` first (Section 10.5).
The workspace shows **live session** instead of b06.	The backend session already holds a design (yours or an agent's). Pick a task from the problem picker, or restart the backend for an empty session.
The front end cannot reach the backend.	The backend listens on `127.0.0.1:8765` and the front end proxies `/api` and `/ws` to it; start the backend first and open `http://127.0.0.1:5173`.

12.4 Reporting a problem

Include the session folder (at least `session.jsonl` and the report of the affected design), the output of the backend console, the AgenticTO version and the GPU model. A log lets the authors replay exactly what happened. The repository's issue tracker will be at `https://github.com/nbbllxx0/From-brief-design-to-verified-release` once the code is public.

A

Specification reference

Every object and field of the specification schema `agentto.spec/v0`. The tables are generated from the live model in `src/agentto/spec.py`, which is what the tools validate against.

All objects reject unknown fields. Vectors are JSON arrays: `[number × 3]` is $[x, y, z]$; `[number × 2]` is a pair in the increasing axis order described in Section 5.2. The backend serves the same schema as JSON Schema at `GET /api/schema`.

> **Exported schema file is out of date.** The tables in this appendix are generated from the live model in `src/agentto/spec.py`, which is what the tools validate against. The exported file `schemas/spec_v0.schema.json` differs from it: PatchRegion.shape is missing from the file. The backend serves the live schema at `GET /api/schema`.

A.1 Spec

The top-level document. Lengths and forces are in the spec's own units; nothing depends on the mesh.

Field	Type	Default	Constraints and notes
`schema_version`	`"agentto.spec/v0"` (fixed)	`"agentto.spec/v0"`	
`name`	string	*required*	
`description`	string	`""`	
`units`	`Units`	`{"length": "mm", "force": "N"}`	
`domain`	`BoxDomain`	*required*	
`material`	`Material`	`{"name": "steel", "E": 210000.0, "nu": 0.3}`	
`volume_fraction`	number	*required*	> 0; < 1; fraction of the design (non-passive) volume
`supports`	list of `Support`	*required*	at least 1
`loads`	list of `Load`	*required*	at least 1
`passive`	list of `Passive`	–	
`resolution`	`Resolution`	*required*	

`min_feature`	number or `null`	`null`	> 0; smallest member the user needs, length units
`meta`	object	–	

A.2 `Units`

Length and force units. They label the numbers; switching units does not convert them.

Field	Type	Default	Constraints and notes
`length`	one of `mm`, `m`, `in`	`"mm"`	
`force`	one of `N`, `kN`, `lbf`	`"N"`	

A.3 `BoxDomain`

The design domain: an axis-aligned box with its minimum corner at the origin.

Field	Type	Default	Constraints and notes
`kind`	`"box"` (fixed)	`"box"`	
`size`	[number $\times$ 3]	*required*	

A.4 `Material`

Isotropic linear-elastic material. E is in force per length squared of the spec units (210 GPa is `210000` in N and mm).

Field	Type	Default	Constraints and notes
`name`	string	`"steel"`	
`E`	number	`210000.0`	> 0; Young's modulus in force/length^2 of the spec units
`nu`	number	`0.3`	> -1; < 0.5

A.5 Support

Fixes the nodes of a region in the listed directions. When `fix` is omitted, all three directions are fixed; `["y", "z"]` makes a roller free in x.

Field	Type	Default	Constraints and notes
`id`	string	*required*	
`region`	region: `face`, `patch`, `edge`, `box`, `cylinder`	*required*	
`fix`	list of one of `x`, `y`, `z`	–	at least 1
`note`	string	`""`	

A.6 Load

A total force shared equally by the nodes of a region, so the total does not change with the mesh.

Field	Type	Default	Constraints and notes
`id`	string	*required*	
`region`	region: `face`, `patch`, `edge`, `box`, `cylinder`	*required*	
`force`	[number × 3]	*required*	
`distribution`	`"uniform_nodes"` (fixed)	`"uniform_nodes"`	
`note`	string	`""`	

A.7 Passive

A keep-in (`solid`) or keep-out (`void`) volume. Elements whose centres fall inside are held at that state and excluded from the volume fraction.

Field	Type	Default	Constraints and notes
`id`	string	*required*	
`kind`	one of `solid`, `void`	*required*	
`region`	region: `box`, `cylinder`	*required*	
`note`	string	`""`	

A.8 Resolution

The base mesh. Give exactly one of the two fields.

Field	Type	Default	Constraints and notes
`h`	number or `null`	`null`	> 0
`cells_short_edge`	integer or `null`	`null`	≥ 2

A.9 FaceRegion

A whole face of the domain. Valid for supports and loads.

Field	Type	Default	Constraints and notes
`kind`	`"face"` (fixed)	`"face"`	
`face`	one of `x-`, `x+`, `y-`, `y+`, `z-`, `z+`	*required*	

A.10 PatchRegion

A rectangle on a face, or the ellipse inscribed in it. `center` and `size` are in the face's two in-plane axes in increasing order: (y, z) on x faces, (x, z) on y faces, (x, y) on z faces. Valid for supports and loads.

Field	Type	Default	Constraints and notes
`kind`	`"patch"` (fixed)	`"patch"`	
`face`	one of `x-`, `x+`, `y-`, `y+`, `z-`, `z+`	*required*	
`center`	[number × 2]	*required*	
`size`	[number × 2]	*required*	full widths along the face's two in-plane axes
`shape`	one of `rect`, `ellipse`	`"rect"`	

A.11 EdgeRegion

A grid line parallel to `axis` through the point `at` (the other two coordinates, increasing axis order). Valid for supports and loads.

Field	Type	Default	Constraints and notes
`kind`	`"edge"` (fixed)	`"edge"`	
`axis`	one of `x`, `y`, `z`	*required*	
`at`	[number × 2]	*required*	
`span`	[number × 2] or `null`	`null`	optional [start, end] along the axis; default: full length

A.12 BoxRegion

An axis-aligned box. Valid for supports, loads and keep-in/keep-out regions.

Field	Type	Default	Constraints and notes
`kind`	`"box"` (fixed)	`"box"`	
`min`	[number × 3]	*required*	
`max`	[number × 3]	*required*	

A.13 CylinderRegion

A solid circular cylinder. Valid for supports, loads and keep-in/keep-out regions.

Field	Type	Default	Constraints and notes
`kind`	`"cylinder"` (fixed)	`"cylinder"`	
`axis`	one of `x`, `y`, `z`	*required*	
`center`	[number × 2]	*required*	axis position in the other two axes (increasing axis order)
`radius`	number	*required*	> 0
`span`	[number × 2]	*required*	

A.14 Validation beyond the field types

The model also checks, when a document is loaded:

- every `id` is unique across `supports`, `loads` and `passive`;
- a box has `min` < `max` on every axis, and a cylinder span has start < end;
- every domain size is positive;
- a resolution gives exactly one of `h` and `cells_short_edge`.

Everything else, such as whether a region selects any nodes or whether the supports remove the rigid-body modes, is the job of the rules (Chapter 6).

B

Tool reference

All 27 tools of the contract, with their arguments, types and defaults. The entries are generated from the tool registry in `src/agentto/tools.py`; the descriptions are the tools' own docstrings, which are also what an agent sees over MCP.

Every tool is called as `Session.call(name, args, actor)` from Python, as an MCP tool of the same name, or by the workspace. Arguments are validated against the signature; unknown arguments are rejected. A call returns `{"ok": true, "seq", "spec_hash", "result"}` or `{"ok": false, "seq", "spec_hash", "error"}`. Tools marked "changes the spec" are undoable; a failed call leaves the specification unchanged. Profiles are explained in Section 9.2.

B.1 Build the spec

`new_spec``(name, size, length_unit, force_unit)` — profiles: spec, clarify, full; changes the spec (undoable)

Start a new spec: a box design domain of the given size (x, y, z) with its min corner at the origin. Replaces any current spec. Material defaults to steel (E 210e3, nu 0.3) until set_params changes it.

Argument	Type	Default
`name`	string	*required*
`size`	[x, y, z]	*required*
`length_unit`	`mm` | `m` | `in`	`"mm"`
`force_unit`	`N` | `kN` | `lbf`	`"N"`

`set_spec``(spec)` — profiles: spec, clarify, full; changes the spec (undoable)

Replace the whole spec with an agentto.spec/v0 document. A document that does not pass the schema yet (e.g. no load) is kept as a draft and its schema errors are returned.

Argument	Type	Default
`spec`	object	*required*

`load_template``(name, k)` — profiles: full; changes the spec (undoable)

Load a FEAD benchmark problem (2:1:1 box, E = 1) at k elements on the short edge.

Argument	Type	Default
`name`	`cantilever` | `bridge` | `torsion`	*required*
`k`	integer	`40`

set_params(name, description, size, length_unit, force_unit, E, nu, material_name, volume_fraction, h, cells_short_edge, min_feature, clear_min_feature) profiles: spec, clarify, full; changes the spec (undoable)

Set scalar spec fields; only the arguments given change. E is in force/length ^ 2 of the spec units (210 GPa = 210e3 N/mm ^ 2). volume_fraction is of the design (non-passive) volume, 0-1. Resolution: give h (element edge, length units) or cells_short_edge, not both.

Argument	Type	Default
name	string, optional	–
description	string, optional	–
size	[x, y, z], optional	–
length_unit	mm | m | in, optional	–
force_unit	N | kN | lbf, optional	–
E	number, optional	–
nu	number, optional	–
material_name	string, optional	–
volume_fraction	number, optional	–
h	number, optional	–
cells_short_edge	integer, optional	–
min_feature	number, optional	–
clear_min_feature	boolean	false

add_support(id, region, fix, note) profiles: spec, clarify, full; changes the spec (undoable)

Fix the nodes of a region in the given directions. Regions: face {face}; patch {face, center, size, shape rect|ellipse} with center/size in the face's two in-plane axes in increasing order ((y,z) on x faces, (x,z) on y faces, (x,y) on z faces); edge {axis, at, span?}; box {min, max}; cylinder {axis, center, radius, span}. Coordinates are absolute, origin at the domain's min corner.

Argument	Type	Default
id	string	*required*
region	region (face, patch, edge, box, cylinder)	*required*
fix	list of x | y | z	["x", "y", "z"]
note	string	""

add_load(id, region, force, note) profiles: spec, clarify, full; changes the spec (undoable)

Apply a total force vector (spec force units, x y z components) shared equally by the region's nodes. Region forms as in add_support.

Argument	Type	Default
id	string	*required*
region	region (face, patch, edge, box, cylinder)	*required*
force	[x, y, z]	*required*
note	string	""

add_passive(id, kind, region, note) profiles: spec, clarify, full; changes the spec (undoable)

Keep-in (solid) or keep-out (void) volume, a box or cylinder: elements whose centres fall inside are held at that state and excluded from the volume fraction.

Argument	Type	Default
`id`	string	*required*
`kind`	`solid` | `void`	*required*
`region`	region (box, cylinder)	*required*
`note`	string	`""`

`update_item``(id, region, fix, force, kind, note, new_id)` profiles: spec, clarify, full; changes the spec (undoable)

Change fields of an existing support, load or passive region (only the arguments given).

Argument	Type	Default
`id`	string	*required*
`region`	object, optional	–
`fix`	list of `x` | `y` | `z`, optional	–
`force`	[x, y, z], optional	–
`kind`	`solid` | `void`, optional	–
`note`	string, optional	–
`new_id`	string, optional	–

`remove_item``(id)` profiles: spec, clarify, full; changes the spec (undoable)

Delete a support, load or passive region by id.

Argument	Type	Default
`id`	string	*required*

`undo``()` profiles: spec, clarify, full; changes the spec (undoable)

Revert the last spec change.

No arguments.

`redo``()` profiles: spec, clarify, full; changes the spec (undoable)

Re-apply the last spec change that undo reverted.

No arguments.

B.2 Inspect and check

`get_spec``()` profiles: spec, clarify, full

The current spec document (possibly incomplete).

No arguments.

`list_regions``(h)` profiles: spec, clarify, full

Ground the spec on the mesh: domain faces and their in-plane axes, then every support/load/passive item with its physical bounding box and size, and how many nodes (or

elements) it selects at mesh size h (default: the spec's resolution), with any snapping. An item that selects 0 nodes does nothing.

Argument	Type	Default
h	number, optional	–

validate_spec(h) profiles: spec, clarify, full

Run the schema and the deterministic rule base (well-posedness, rigid-body modes, empty regions, loads on fixed nodes, overlaps, resolvable minimum feature, memory) at mesh size h (default: the spec's). Returns ok plus findings with stable codes and suggested fixes. Only an ok spec can be solved.

Argument	Type	Default
h	number, optional	–

explain_spec() profiles: spec, clarify, full

Plain-language summary of the spec, for confirming intent with the user.

No arguments.

render(view) profiles: spec, clarify, full

PNG picture of the domain with supports (blue), load arrows (red) and keep-in/keep-out regions, drawn from the spec geometry. Use it to confirm the setup with the user before solving.

Argument	Type	Default
view	iso | front | top | side | back_iso	"iso"

ladder() profiles: spec, clarify, full

Resolution rungs for this spec (sketch, preview, refine, release; each 2x finer per axis). The spec's resolution h is the base mesh the ladder is built from, not the release mesh: release is the finest rung that fits on the GPU, often finer than the spec's h. Only a solve at the release rung can be released.

No arguments.

B.3 Ask the user

ask_clarification(question, options, previews, about) profiles: clarify, full

Ask the user ONE question when the brief is ambiguous in a way that changes the design (which face, how large, what force or material). Give 2-4 options, each a concrete reading: the full item the spec would contain (item_kind + item, as you would pass to add_support / add_load / add_passive) and/or parameter values (params, e.g. {"volume_fraction": 0.3}). Returns the chosen option, which you then apply with the editing tools, or choice null with the user's comment. previews=true (full profile) first solves each reading at the sketch rung, so the user sees what it does to the design. about: the spec items the question concerns (keys as for set_source); in a provenance session every one must be tagged assumed.

Argument	Type	Default
`question`	string	*required*
`options`	list of options (see below)	*required*
`previews`	boolean	`false`
`about`	list of string, optional	–

Clarification options. Each element of `options` is an object with these fields (`ClarifyOption`); unknown fields are rejected.

Argument	Type	Default
`label`	string	*required*
`item_kind`	`support` | `load` | `passive`, optional	–
`item`	object, optional	–
`params`	object, optional	–

B.4 Record value sources

`set_source``(key, source, note)` profiles: full

Record where one spec item's value comes from. key: units, domain, material, volume_fraction, resolution, or support:<id>, load:<id>, passive:<id>. source: stated (the brief gives it), derived (computed from numbers the brief gives), assumed (your choice where the brief is silent or ambiguous), answered (fixed by an answer to ask_clarification). note: what was stated, derived or assumed.

Argument	Type	Default
`key`	string	*required*
`source`	`stated` | `derived` | `assumed` | `answered`	*required*
`note`	string	`""`

`list_sources``()` profiles: full

Every spec item with its source tag (stated, derived, assumed, answered), the untagged items, and the items tagged assumed.

No arguments.

B.5 Solve

`solve``(rung, h, warm, optimize, wait, timeout_s)` profiles: full

Optimize the current spec on a ladder rung (or mesh size h). Warm-starts from this spec's design on the 2x coarser grid when there is one. By default that warm start only certifies the upsampled coarse design (about 12 iterations); optimize=True re-opens it (about 60 iterations, ~60% of a cold run, compliance within +0.1% of cold or better on the X1 problems). wait=False returns the job id at once. The result carries the solver's certificate; only a certified release is a released design.

Argument	Type	Default
rung	sketch | preview | refine | release, optional	–
h	number, optional	–
warm	boolean	true
optimize	boolean	false
wait	boolean	true
timeout_s	number	900

status() profiles: full

Jobs of this session, newest last: rung h, status, iteration, certified.

No arguments.

wait_job(job, timeout_s) profiles: full

Wait for a queued or running solve (default: the newest job) and return its result, as solve(wait=True) would. Use this after a solve timed out instead of cancelling and resubmitting.

Argument	Type	Default
job	string, optional	–
timeout_s	number	900

cancel() profiles: full

Stop the running solve (it ends with status cancelled at the next iteration).

No arguments.

field_stats(job, region, box_min, box_max) profiles: full

Numbers on a finished design (default: the latest finished job): material fraction, grey fraction, solid components, and whether each load is connected through solid to a support. With region (an item id) or box_min/box_max, also the density inside that region.

Argument	Type	Default
job	string, optional	–
region	string, optional	–
box_min	[x, y, z], optional	–
box_max	[x, y, z], optional	–

B.6 Export

export_stl(job, pool) profiles: full

Write the design's rho = 0.5 surface as a binary STL (default: the latest finished job; pool > 1 averages pool ^ 3 element blocks first). Only a certified release is a released design: anything else is exported as a draft and says so.

Argument	Type	Default
`job`	string, optional	–
`pool`	integer	1

`export_report``(job)` profiles: full

Write the design report (JSON and Markdown): spec and hash, rung and solver, iterations and time, compliance, every certificate check, the STL if one was exported, and the hash of this session's log.

Argument	Type	Default
`job`	string, optional	–

C

Certificate, report and log fields

The fields of the solver's certificate record, of the design report written at export, and of the records in the session log. The certificate and report tables are generated from `solver.py` and `export.py`.

C.1 Certificate checks

Check (record key)	Result-card label	Predicate	Passes when
`converged`	converged	A_{conv}	The optimizer met its stopping criterion within the iteration cap. code: `bool(res.get('converged'))`
`state_solve_accepted`	FP64 state solve accepted	A_{res}	The final state solve was accepted on the true relative residual, recomputed in FP64, at the configured tolerance τ. code: `rel is not None and rel <= self.cfg.tol`
`no_vram_spill`	no VRAM spill	$A_{\text{no spill}}$	No growth of shared GPU memory beyond the spill limit was observed: the solve stayed in dedicated GPU memory. code: `not self.spill.spilled`
`spill_monitored`	spill monitored	A_{monitor}	Spill sampling ran during the iterations. Only the release rung enables it, so lower rungs fail this check and are never certified. code: `self.cfg.spill_check_every > 0`
`volume_ok`	volume within 0.1%	A_{vol}	The largest recorded volume error is at most 10^{-3} (0.1%). code: `res.get('max_vol_error') is not None and res['max_vol_error'] <= 0.001`
`passive_exact`	keep-in/out exact	A_{passive}	Every keep-in element has physical density exactly 1 and every keep-out element exactly 0. code: `bool(pc.get('keep_in_ok', True) and pc.get('keep_out_ok', True))`

C.2 Certificate record

The solver attaches the full certificate to the worker's *done* event, which the backend forwards to the workspace. The session log, the tool responses and the design report keep the verdict (`certified`) and the six `checks`.

Field	Meaning
`certified`	true only when every check passes
`checks`	the six predicates above
`rel_true`	final true relative residual (FP64)
`tol`	acceptance tolerance τ
`solver`	state solver (`mg64` or `mg_ir`)
`coarse_bc`	coarse support rule (`inject` or `dilate`)
`grid`	elements per axis
`n_elem`	number of elements
`h`	element edge, spec length unit
`units`	the spec's length and force units
`compliance`	solver compliance (nondimensional)
`compliance_phys`	compliance in force $\times$ length
`volume_phys`	design volume fraction reached
`passive`	keep-in minimum and keep-out maximum density
`iterations`	optimization iterations
`termination`	why the optimizer stopped
`stopping`	the stopping settings (`conv_drho`, `conv_dc_rel`, `conv_window`, `conv_moving_ppm`)
`regularization`	release element edge beside the stated one, their ratio, the filter radius in elements and as a length at both sizes, the continuation schedule, the warm-start grid and the iterations on the release grid (recorded; the engineer does not accept them)
`issued_at`	local time stamp

C.3 Design report

`export_report` writes these fields to ⟨*name*⟩_⟨*job*⟩_`report.json` and renders the same content as Markdown.

Field	Meaning
`report`	format tag
`issued_at`	time stamp
`spec_hash`	hash of the normalized spec
`spec`	the full spec
`job`	job id
`h`	element edge
`grid`	elements per axis
`n_elem`	number of elements
`solver`	state solver
`warm_used`	whether the solve was warm-started
`iterations`	optimization iterations
`compute_s`	solver compute time, s
`compliance_phys`	compliance in spec units

`volume`	volume fraction reached
`status`	job status
`certified`	certificate verdict
`checks`	every certificate check
`release_rung`	whether the solve ran on the release rung
`regularization`	the certificate's regularization record (release and stated element edge, filter length, schedule)
`stl`	STL file record (path, triangles, bytes, SHA-256, closedness)
`session_log_sha256`	SHA-256 of `session.jsonl` at export time

C.4 Solve result

Every `solve` that finishes, and every `result` record in the log, contains: `job`, `status` (`converged`, `not_converged`, `cancelled`, `solve_failed`, `failed`, `error` or `skipped`), `h`, `grid`, `n_elem`, `solver`, `warm_used`, `warm_from_seq`, `warm_kind` (`coarser` or `same_grid`), `iterations`, `compliance_phys`, `volume`, `compute_s`, `wall_s`, `certified`, `checks`, `release_rung`, `spec_hash` and `preview` (true for a clarification preview), plus `coarse_pad` (the coarse-level policy) and, after a solver retry, `retry`.

C.5 Log records

Type	Fields
`session`	`t`, `agentto` (version), `schema`, `id`, `profile`
`call`	`seq`, `t`, `actor`, `tool`, `args`, `ok`, `result` or `error`, `spec_hash`, `dt_s`, optional `meta`
`result`	`seq` (of the solve call), `t`, `job`, `result` (the solve result above)
`answer`	`seq` (of the question), `t`, `answer` (`choice`, `label`, `by`, `comment`), optional `previews`

D

Keyboard, mouse and command palette

Shortcuts of the workspace. The key table is generated from the keyboard map in `web/src/components/Overlays.tsx`, the same list that ? shows.

D.1 Keys

Shortcuts are off while you type in a field.

Key	Action
V	Select
S	Support tool: click a face
L	Load tool: click a face
K	Keep-in tool: click a face
O	Keep-out tool: click a face
Esc	Cancel the tool, close this
Ctrl + Z	Undo
Ctrl + Shift + Z or Ctrl + Y	Redo
Ctrl + K	Command palette
?	This keyboard map
Enter or Shift + Enter	In the assistant box: send / new line

D.2 Mouse

Action	Effect
Left-drag in the viewport	Orbit
Right-drag	Pan
Wheel	Zoom
Click a face of the orientation cube	Look along that axis
Click an item (viewport or tree)	Select it; the inspector shows its values
Click a face with a tool armed	Add a support, load, keep-in or keep-out region there
Hover a timeline record	Show its time and details
Hover a proposal card	Draw its ghost in the viewport

D.3 Command palette

Ctrl + K opens the palette (Figure D.1). Type to filter, move with the arrow keys and press Enter to run a command; Esc closes it. Only commands that can run now are listed: run any rung, **Optimize release** (once the rung below the release has converged), **Stop the running solve**, **Show the surface of the last design**, **Export STL and report**, **Undo**, **Redo**, the five tools, and the theme switch.

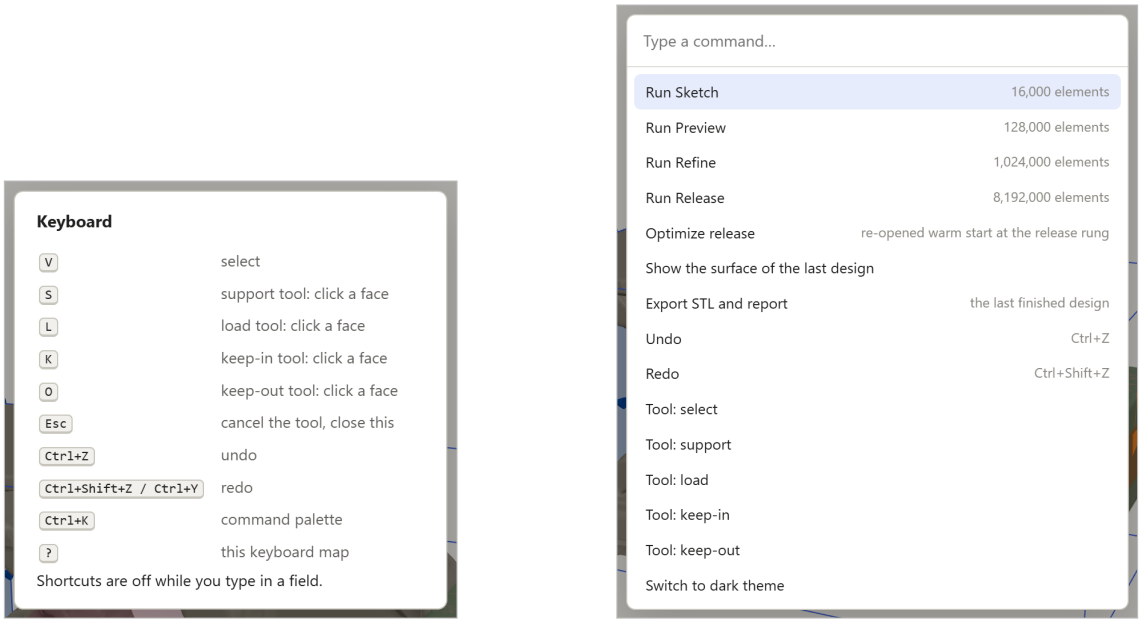


Figure D.1. Left: the keyboard map (?). Right: the command palette (Ctrl + K).

Script catalogue

Every script in `scripts/`, with the first paragraph of its docstring and its usage line. The table is generated from the module docstrings; read the full docstring before running a script. Chapter 11 says which scripts need the GPU or a language model.

All scripts run from the repository root with the Python of the `agentto` environment, and put `src/` on the path themselves. The `-B` flag keeps Python from writing bytecode caches. Scripts that solve need `AGENTTO_VOXFEM_SRC` (Section 2.4).

Script	What it does (from its docstring)
`apply_adjudication.py`	Read the author's filled adjudication sheet back and apply what can be applied mechanically. `python -B scripts/apply_adjudication.py <filled ADJUDICATION_SHEET.xlsx> [--dry]`
`apply_adjudication_followup.py`	Read the filled follow-up adjudication sheet back (scripts/make_adjudication_pack_followup.py) and apply it. `python -B scripts/apply_adjudication_followup.py <filled ADJUDICATION_SHEET_FOLLOWUP.xlsx> [--dry]`
`b19_release_aware.py`	b19's 36M release under the pad-aware coarse rule (results/solver/NOTES.md), warm from the design X3's own refine rung produced, with the same reopen_long schedule X3's release used. X3's release under voxfem's rule: 61 iterations, 1507.6 s compute, C 121.26051525896641, certified. `python -B scripts/b19_release_aware.py [aware|voxfem]`
`c4_gate_ablation.py`	C4.3: the release contract with each gate switched off. `python -B scripts/c4_gate_ablation.py [census|rules_off]`
`c4b_void_energy.py`	C4b: would a certificate predicate on the solved field catch a load inside a keep-out region? `python -B scripts/c4b_void_energy.py`
`c4c_fault_matrix.py`	C4c: which specification faults do the deterministic rules catch? `python -B scripts/c4c_fault_matrix.py`
`c4d_assumption_flags.py`	C4d: does the stored specification itself expose a wrong reading without the reference?
`c4e_modulus_plausibility.py`	C4e: would a deterministic modulus-plausibility check, which the rule set does not contain, flag wrong readings?
`c4f_value_traceability.py`	C4f: a back-filled 'stated in the brief' tag per value, computed without the reference reading.
`codex_mcp_probe.py`	Week-3 probe: one grounded (condition G) Codex session over MCP, to check the tool contract end to end and measure tokens and tool calls. `python -B scripts/codex_mcp_probe.py [b01_cantilever_block]`

codex_probe.py	Week-1 probe: one text-only (condition T) session on Codex CLI, to check headless use and measure tokens.
compare_adjudications.py	Compare the second-round adjudication (S.Y.) with the first (J.W.), and bound the scores. `python -B scripts/compare_adjudications.py <filled ADJUDICATION_SHEET_R2.xlsx>`
diag_warm_b06.py	Diagnose warm-start convergence and iteration cost on brief b06 (motor mount, passive bosses).
fill_sheet_from_txt.py	Copy the adjudicator's plain-text results into a copy of ADJUDICATION_SHEET.xlsx, for apply_adjudication.py. `python -B scripts/fill_sheet_from_txt.py <results.txt> <out.xlsx>`
followup_equivalence.py	Which contested follow-up specifications state the same problem (benchmark comparator, agentto.bench.equivalent), so the adjudication sheet can point a repeated reading to its first row. No GPU, no LLM.
gui_screenshots.py	Paper screenshots of the GUI, driven through headless Chrome over the DevTools protocol (no extra packages). `python -B scripts/gui_screenshots.py [overview] [assistant]`
gui_test.py	End-to-end GUI test: headless Chrome over the DevTools protocol, real mouse and keyboard input, small meshes. `python -B scripts/gui_test.py [setup] [ladder] [overlays] [assistant]`
guide_shots.py	Screenshots for the user guide (docs/USER_GUIDE.md): a real session driven through headless Chrome with real clicks and keys, on the same harness as scripts/gui_test.py. `python -B scripts/guide_shots.py [build] [solve] [overlays] [assistant] [attach]`
latency_check.py	Week-4 latency check against the GUI latency budget (GUI_INTERACTION_PLAN.md section 6), through the running backend exactly as the GUI uses it (WebSocket tool calls and events, HTTP for meshes). `python -B scripts/latency_check.py [b06_motor_mount]`
levels_check.py	Per-iteration cost of b01 at its refine (1 mm, 2M) and release (0.5 mm, 16M) rungs with the session's multigrid levels (rules.session_levels): 10 cold iterations each, s/iteration printed. Checks the cost model.
make_adjudication.py	Human adjudication pack: everything the author should check by hand before the numbers go into the paper. `python -B scripts/make_adjudication.py writes paper/adjudication/`
make_adjudication_pack.py	Adjudication package for the author: one Excel fill sheet plus the evidence, zipped. `python -B scripts/make_adjudication_pack.py`
make_adjudication_pack_followup.py	Adjudication package for the follow-up X2 runs, so their contested outputs get the same judgement as the suite's. `python -B scripts/make_adjudication_pack_followup.py`
make_adjudication_pack_gnr.py	Adjudication package for the rule-free ablation (GNR), so its outputs get the same adjudication as G's. `python -B scripts/make_adjudication_pack_gnr.py`
make_adjudication_pack_r2.py	Blinded second-round adjudication package (second adjudicator: S.Y.; first round: J.W.). `python -B scripts/make_adjudication_pack_r2.py`
make_briefs.py	Writes the X2 task suite: briefs b01-b50 and their intended readings. Re-run to regenerate.
make_figures.py	Paper figures, generated from the result files only (no GPU, no LLM). `python -B scripts/make_figures.py writes results/figures/<name>.pdf and .png`

make_ledger.py	Generated claims ledger: every number the paper may state, read from the result files, with its source and status. python -B scripts/make_ledger.py writes results/claims_ledger.md (no GPU, no LLM)
make_release.py	Release package for the paper: what is published, its size and checksum, a scrub of private paths, and a privacy scan of the bytes that would be published. python -B scripts/make_release.py writes paper/release/MANIFEST.md and paper/release/files.tsv
make_renders.py	3D renders of released designs for the paper (pyvista, off screen). python -B scripts/make_renders.py [gallery|arms|x5]
make_tables.py	Paper tables, generated from the result files only (no GPU, no LLM). python -B scripts/make_tables.py writes paper/tables/<name>.md and <name>.tex (booktabs)
make_user_guide.py	Build the user guide: the numbered overview figure, docs/USER_GUIDE.html and docs/AgenticTO_User_Guide.pdf. python -B scripts/make_user_guide.py
mg_padding_study.py	How void padding of the multigrid grid costs PCG iterations (finding of X5 and X3 b19, 2026-10-30). python -B scripts/mg_padding_study.py [voxfem|aware]
rerun_113m_guarded.py	Clean re-run of the 113M escalation chain (pilot chain B) with voxfem's guards ON.
serve.py	Start the agentto backend (127.0.0.1:8765) without installing the package. python -B scripts/serve.py [--ladder-max 1.2e7] (a lower ladder cap, for GUI tests on small meshes) python -B scripts/serve.py [--ladder-max 1.2e7]
validate_suite.py	Task-suite check on the GPU: every solve/clarify intended spec is optimized cold at its preview rung (the rung x2_score uses for the consequence-weighted error), in mm and N. A task whose preview does not converge, or does not certify, cannot give a reliable CWE reference and is flagged. python -B scripts/validate_suite.py [b11 b12 ...]
x0_coarse_bc.py	X0 general BCs: voxfem's injection coarse rule vs RobustGMG's dilation on briefs whose supports injection loses.
x0_crosscheck_skfem.py	X0 independent cross-check, solve side: recompute the compliance of each exported design with scikit-fem.
x0_export_crosscheck.py	X0 independent cross-check, export side: optimise two problems with passive regions, then save the final design's moduli, supports and nodal loads so another FE code can recompute the compliance at that fixed design.
x0_identity_gpu.py	X0 identity (GPU part): voxfem.SIMP on the spec-built problem vs on voxfem's own problem.
x0_replay.py	X0 replay test: a scripted session through the tool contract, then a replay of its log with no LLM. python -B scripts/x0_replay.py [b01_cantilever_block b06_motor_mount ...]
x1_escalation.py	X1: certified coarse-to-fine escalation on the GUI's own ladder (preview -> refine -> release). python -B scripts/x1_escalation.py [problem ...]
x1_fix_cold.py	X1 follow-up: re-run a failed cold reference at the release rung with a more robust solver, and re-score its arms. python -B scripts/x1_fix_cold.py bridge

x1_refine_pilot.py	X1 pilot 2: do any warm arms actually refine? One coarse -> fine doubling, all arms in agentto.escalation.
x1_warm_arms_pilot.py	X1 pilot: warm-start arms on one coarse -> fine step, against a cold fine run.
x2_failure_categories.py	X2: failure categories of the first-repeat text-only (T) and grounding (G) outputs, and of the T outputs that G corrected or broke. Categories, in order: right, did not refuse (refusal brief), refused a solvable brief, no specification, schema-invalid, rule error, wrong reading (schema-valid, rule-clean, not the reference).
x2_gnr_ablation.py	X2 ablation GNR: the grounding protocol G with the rule findings removed from validate_spec (same prompt, tools, provider and CLI), first repeat, scored like every other X2 condition against the adjudicated references. Compares GNR with G task by task. No LLM calls, no solves.
x2_provenance.py	X2 representation and schema-feedback arms, first repeat, scored like every other condition against the adjudicated references. No LLM calls, no solves.
x2_rescore.py	Rescore every saved X2 run against the current intended readings (after the author's adjudication). `python -B scripts/x2_rescore.py [--dry]`
x2_rules_baseline.py	X2 condition R: a deterministic rule-based brief parser (no LLM), the baseline the plan names. `python -B scripts/x2_rules_baseline.py`
x2_run.py	X2 harness: brief -> spec under one condition, on the subscription Codex CLI only (llm/cli.py refuses API keys). `python -B scripts/x2_run.py --cond G [--briefs b01 b06] [--rep 1] [--force] [--provider codex|claude]`
x2_score.py	X2 consequence-weighted error (CWE), on the GPU: what a wrong spec costs in the design. `python -B scripts/x2_score.py [T G claude_high/GC ...] [--force]`
x3_regularization.py	Certificate regularization block reconstructed from the logs of the complete agent sessions (X3).
x3_retime_aware.py	Re-time the X3 sessions under the pad-aware coarse rule (results/solver/NOTES.md), without any LLM. `python -B scripts/x3_retime_aware.py [b01 ...]`
x3_run.py	X3: end-to-end sessions. An agent takes a brief to a certified release and its STL + report, alone. `python -B scripts/x3_run.py b01_cantilever_block [b06_motor_mount ...] [--provider codex|claude]`
x5_chain.py	X5 addendum: the ladder's own release path against a cold run at the finest rung. `python -B scripts/x5_chain.py [problem ...]`
x5_connectivity_fullres.py	X5: connectivity of the binarized span-bridge designs at full resolution (the fields rendered in Fig. S2).
x5_resolution.py	X5: why 10^8 elements. Does a coarser optimum stay competitive when the part is judged at fine resolution? `python -B scripts/x5_resolution.py [problem ...]`
x6a_latency.py	X6a: the GUI latency budget (GUI_INTERACTION_PLAN.md section 6) over the 14 released X3 tasks, through the running backend exactly as the GUI drives it (WebSocket tool calls and events, HTTP for meshes). `python -B scripts/x6a_latency.py [--repeats 20] [b01_cantilever_block ...]`
x6b_klm.py	X6b: interaction cost of setting up and releasing the 15 X3 tasks, by Keystroke-Level Model (no participants). `python -B scripts/x6b_klm.py writes results/X6/klm.json and results/X6/klm_summary.md`

`x6c_cards.py`	X6c: agent action cards and validation catches under the scripted oracle (no LLM, no GPU; reads the X2 runs). `python -B scripts/x6c_cards.py writes results/X6/cards.json and results/X6/cards_summary.md`

F

Glossary

Act mode
The assistant edits the live specification directly; a whole run can be undone in one step.

Actor
Who made a logged call: `user`, `agent`, `system` or `script`.

Admissible reading
An interpretation of a brief, other than the intended one, that the adjudication accepted as correct.

Brief
The written design requirement in natural language.

Card
One proposed change (add, change, remove, set) returned by the assistant in Propose mode.

Certificate
The solver-issued record of six numerical acceptance predicates. Certified means all six pass.

Clarification
A question with two to four concrete readings, asked when a brief leaves a design-changing fact open.

Cold start
A solve from uniform density with the full continuation schedule.

Compliance
$C = f^{\mathsf{T}}u$, the work done by the loads; lower is stiffer. Reported in force × length of the spec units.

Consequence preview
The sketch-rung design and compliance of one clarification option, shown on its chip.

Continuation schedule
The sequence of penalty p, projection sharpness β, move limit and filter radius during an optimization.

Design volume
The domain minus all keep-in and keep-out elements; the volume fraction refers to it.

FP64
64-bit floating point.

Grounding
Checking a specification against geometric facts returned by tools: selected nodes, bounding boxes, rule findings, pictures.

Keep-in / keep-out region
A box or cylinder held solid (`solid`) or empty (`void`); a *passive* item in the document.

Ladder
The sequence of grids, each twice as fine per axis as the one below, on which a specification is solved.

Long re-opening
The warm release policy behind **Optimize release**: the cold schedule from its second phase.

MCP
Model Context Protocol, the interface through which agents call the tools.

`mg64`, `mg_ir`
The FP64 multigrid state solver and its mixed-precision iterative-refinement variant.

Oracle
The scripted answerer of the benchmark, which answers clarification questions from the stored reference reading.

Profile
The tool set an MCP server offers: `spec`, `clarify` or `full`, or the provenance variants `spec_prov` and `clarify_prov`.

Propose mode
The assistant works on a private copy and returns its changes as cards.

Release rung
The finest rung of the ladder; only a certified result on it is a released design.

Replay
Re-running a session log without a language model, checking that every result reproduces.

Rule
A deterministic check of a specification with a stable code (R001–R013).

Rung
One grid of the ladder: sketch, preview, refine or release.

Silent wrong
A valid, non-refused specification that does not match the intended or an admissible reading.

SIMP
Solid isotropic material with penalization, the density-based topology optimization method used.

Specification
The typed JSON document `agentto.spec/v0` that states the design problem.

Spec hash
The first 16 hex digits of the SHA-256 of the normalized specification.

Spill
GPU memory moved into shared system memory by the driver; the spill guard stops a solve when it happens.

Stale
A result whose specification hash no longer matches the current specification.

Tail
The default warm release policy: the final continuation phase only. It certifies the transferred design.

`voxfem`
The GPU finite-element and SIMP package from the authors' companion work that AgenticTO runs on; included in `src/voxfem/` under the BSD 3-Clause License.

Warm start
A solve that starts from an earlier design: upsampled, filtered once and volume-matched.

Quick reference

Start

`python -B scripts/serve.py`
`node web/node_modules/vite/bin/vite.js web`
open `http://127.0.0.1:5173`
small GPU: `serve.py --ladder-max 1.2e7`
solver: `AGENTTO_VOXFEM_SRC=`⟨*dir*⟩

Workflow

1. State: S support, L load, K keep-in, O keep-out, then click a face; edit in the inspector.
2. Check: fix every red finding.
3. Solve: **Sketch** → **Preview** → **Refine** → **Optimize release**.
4. Accept: card reads **certified**, grid is the release grid.
5. Export: **Export STL + report**.

Certified = all six

converged · FP64 state solve accepted ($r_{\text{true}} \le \tau$) · no VRAM spill · spill monitored (release rung only) · volume within 0.1% · keep-in/out exact

Release policies

Release: tail, ≈12 it., certifies the upsampled design.
Optimize release: long re-opening, ≈60 it., re-optimizes.
after an edit: short re-opening on the same grid.

Keys

V select Esc cancel ? keymap
Ctrl + K palette Ctrl + Z undo
Ctrl + Shift + Z / Ctrl + Y redo

Rules (errors block solving)

R001 no grid fits
R002 support selects nothing
R003 rigid-body mode free
R004 load empty or zero
R005 load on fixed DOFs (err >50%)
R006 no design elements
R007 keep-in overlaps keep-out
R008 item inside keep-out
R009 min feature too small (warn)
R010 region snapped (warn)
R011 passive region empty (warn)
R012 coarse multigrid (warn/info)
R013 GPU memory

Agents (MCP)

```
python -B -m agentto.mcp_server
  --attach http://127.0.0.1:8765
  or --session DIR
  --profile spec|clarify|full
  [--seed spec.json] [--answer-url URL]
```

Codex: `default_tools_approval_mode="approve"`
Claude Code: allow `mcp__agentto`
No API keys: subscription logins only.

Tools

build new_spec, set_spec, load_template, set_params, add_support, add_load, add_passive, update_item, remove_item, undo, redo
inspect get_spec, list_regions, validate_spec, explain_spec, render, ladder
ask ask_clarification
solve solve, status, wait_job, cancel, field_stats
export export_stl, export_report

Files

`sessions/`⟨*time*⟩`/session.jsonl` log
`.../fields/` densities (float16)
`.../exports/` STL, report `.md`/`.json`
replay: `agentto.tools.replay(log)`

Index

A

Act mode 37
`add_load` (tool) 67
`add_passive` (tool) 67
`add_support` (tool) 67
`AGENTTO_VOXFEM_SRC` 7
`--answer-url` (MCP flag) 44
`apply_adjudication.py` (script) 78
`apply_adjudication_followup.py` (script) . 78
architecture 3
`ask_clarification` (tool) 69
`--attach` (MCP flag) 43
authority 3

B

`b19_release_aware.py` (script) 78

C

`c4_gate_ablation.py` (script) 78
`c4b_void_energy.py` (script) 78
`c4c_fault_matrix.py` (script) 78
`c4d_assumption_flags.py` (script) 78
`c4e_modulus_plausibility.py` (script) 78
`c4f_value_traceability.py` (script) 78
`cancel` (tool) 71
cards 39
certificate 2
  predicates 33
claims ledger 55
clarification 38
`codex_mcp_probe.py` (script) 78
`codex_probe.py` (script) 79
`compare_adjudications.py` (script) 79
consequence preview 38
coordinate frame 21

D

`diag_warm_b06.py` (script) 79

E

`explain_spec` (tool) 69
export 35
`export_report` (tool) 72
`export_stl` (tool) 71

F

`field_stats` (tool) 71
`fill_sheet_from_txt.py` (script) 79
`followup_equivalence.py` (script) 79

G

`get_spec` (tool) 68
`gui_screenshots.py` (script) 79
`gui_test.py` (script) 79
`guide_shots.py` (script) 79

H

HTTP API 47

K

keep-in region 24
keep-out region 24

L

`ladder` (tool) 69
`latency_check.py` (script) 79
`levels_check.py` (script) 79
`list_regions` (tool) 68
`list_sources` (tool) 70
`load_template` (tool) 66
long re-opening 32

M

`make_adjudication.py` (script) 79
`make_adjudication_pack.py` (script) 79
`make_adjudication_pack_followup.py` (script) 79
`make_adjudication_pack_gnr.py` (script) . . 79
`make_adjudication_pack_r2.py` (script) . . . 79
`make_briefs.py` (script) 79
`make_figures.py` (script) 79
`make_ledger.py` (script) 80
`make_release.py` (script) 80
`make_renders.py` (script) 80
`make_tables.py` (script) 80
`make_user_guide.py` (script) 80
MCP server 43
`mg_padding_study.py` (script) 80

N

new_spec (tool) 66
--no-rules (MCP flag) 44

O

--oracle (MCP flag) 43

P

passive region ... *see* keep-in region, keep-out region
PolicyError 8
profile 44
--profile (MCP flag) 43
Propose mode 37

R

R001 (rule) 27
R002 (rule) 27
R003 (rule) 27
R004 (rule) 28
R005 (rule) 28
R006 (rule) 28
R007 (rule) 28
R008 (rule) 28
R009 (rule) 28
R010 (rule) 29
R011 (rule) 29
R012 (rule) 29
R013 (rule) 29
redo (tool) 68
regions 22
release policy 32
remove_item (tool) 68
render (tool) 69
replay 51
rerun_113m_guarded.py (script) 80
resolution 24
resolution ladder 2, 31
rules 2
  when they run 26
rung 31

S

--seed (MCP flag) 43
serve.py (script) 80
Session 51
--session (MCP flag) 43
session folder 49
session log 2
  records 49
set_params (tool) 67
set_source (tool) 70
set_spec (tool) 66
short re-opening 32
solve (tool) 70
specification 2
  document 20
specification hash 50
stale result 35
status (tool) 71
subscription policy 8

T

tail schedule 32
three checks 1
troubleshooting 59

U

undo (tool) 68
units 21
update_item (tool) 68

V

validate_spec (tool) 69
validate_suite.py (script) 80
viewport navigation 15
volume fraction 24
voxfem 7

W

wait_job (tool) 71
warm start 32

X

x0_coarse_bc.py (script) 80
x0_crosscheck_skfem.py (script) 80
x0_export_crosscheck.py (script) 80
x0_identity_gpu.py (script) 80
x0_replay.py (script) 80
x1_escalation.py (script) 80
x1_fix_cold.py (script) 80
x1_refine_pilot.py (script) 81
x1_warm_arms_pilot.py (script) 81
x2_failure_categories.py (script) 81
x2_gnr_ablation.py (script) 81
x2_provenance.py (script) 81
x2_rescore.py (script) 81
x2_rules_baseline.py (script) 81
x2_run.py (script) 81
x2_score.py (script) 81
x3_regularization.py (script) 81
x3_retime_aware.py (script) 81
x3_run.py (script) 81
x5_chain.py (script) 81
x5_connectivity_fullres.py (script) 81
x5_resolution.py (script) 81
x6a_latency.py (script) 81
x6b_klm.py (script) 81
x6c_cards.py (script) 82